



M256 Unit 5

UNDERGRADUATE COMPUTING

Software development with Java



Component-based
software development

Unit **5**

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007. Second edition 2008.

Copyright © 2007, 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 5478 0

CONTENTS

1	Introduction	5
1.1	Software development goals	5
1.2	The aims of this unit	6
1.3	Studying this unit	6
2	Software quality	7
2.1	Maintainability	7
2.2	Reusability	9
3	Software components	11
3.1	What is a software component?	11
3.2	Engineering components	12
3.3	Object-oriented software components	15
3.4	Larger software components	20
4	Java packages and specifications	21
4.1	Working with packages	21
4.2	Documenting packages	22
4.3	Distributing packages	25
5	Data hiding: access modifiers	26
5.1	Data hiding and classes	26
5.2	Data hiding and packages	29
6	Data hiding: privacy leaks	36
6.1	Compromising integrity	36
6.2	Protecting mutable objects	39
7	Using and producing components	42
7.1	Design with reuse	42
7.2	Design for reuse	46
7.3	A component-based approach	49
7.4	Component-based development in M256	50
8	Summary	52
	Glossary	53
	References	55
	Index	56

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

Think about software you use regularly, such as an email system or a word processor. Does it always work properly? Our guess is that you will answer 'no'. Software often does not do what it is supposed to, in all sorts of ways. For example, it might routinely freeze up or unexpectedly crash. If other kinds of product behaved like this we would be very surprised. Imagine if motorists found that the engines of brand-new cars stopped unexpectedly in mid-journey many times a day and had to be restarted each time.

So the performance of software often disappoints, and worse: there are many examples of large and expensive software systems that have been utter failures (recall the Child Support Agency system and the Passport Agency system, mentioned in *Unit 1*).

Moreover, software is very expensive to develop, and frequently takes much longer than expected to produce. When finally released, it may turn out to be hard to maintain. In short, software often fails to meet the following quality criteria:

- ▶ reliability;
- ▶ usefulness;
- ▶ affordability;
- ▶ timeliness;
- ▶ maintainability.

1.1 Software development goals

In this unit, we outline an approach to software development that is widely believed to be capable of delivering better software. Software development is such a complex process that software developers can probably never hope for perfection. But with the right techniques they may produce software that satisfies criteria such as the above.

This unit temporarily suspends the flow of the development process you are learning about, in order to prepare you for the design work to come in the next few units. In previous units, we discussed how to analyse and specify the requirements for a system, and we began to explore how to construct software that will do what is required. Our first step was to form an initial idea of what classes will be involved in our software. The next stage will be to start designing how objects of these classes will collaborate to deliver the services required of the system. What messages will be sent, and what objects will they be sent to? This is the sort of question we will be asking in *Unit 6*.

In addressing such questions, we will continue to pursue the notion that object-oriented software should resemble the real world, one manifestation of this being that the tasks an object is charged with correspond sensibly to tasks associated with its real-world analogue. For example, just as visiting a hospital ward may reveal how many patients there are on it, a `Ward` object may be charged with disclosing how many `Patient` objects it is linked with. Given this guiding principle, you might think that assigning each object's responsibilities is a fairly automatic process. However, it turns out that this is far from being the case. Generally, there is more than one way each task can be carried out and the developer has to decide which way to adopt.

To make the right choices, developers need clear goals – they need to know what good software is. Above, we mentioned the qualities of reliability, usefulness, affordability, timeliness and maintainability. But the quality of software also encompasses its potential

for reuse – its **reusability** – which is concerned with writing software once and then using it over and over again in similar situations, instead of starting afresh each time.

The approach to all these issues involves the idea of designing software using software components – reusable building blocks from which systems can be assembled, in much the same way as a kitchen can be assembled from prefabricated units.

Using software components also contributes to the solution of another problem – how to build large-scale systems. Most software projects would be far too big a task for any single person working on their own. Using software components allows the work to be shared amongst many developers, each of whom concentrates on particular components.

1.2 The aims of this unit

In this unit, we will:

- ▶ establish what is meant by software quality (Section 2);
- ▶ introduce the concept of components, and consider the characteristics of object-oriented software components (Section 3);
- ▶ enable you to review and extend your understanding of the component-related aspects of Java (Sections 4, 5 and 6);
- ▶ consider the contribution of object-oriented components to the quality of software, and explain some key issues in the production and use of components (Section 7).

1.3 Studying this unit

The unit contains a number of SAQs and paper-based exercises which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

Section 2 is a short reading-based section. An exercise in this section requires you to consult the Hospital System requirements document in the *Handbook*.

Sections 3 and 7 are longer reading-based sections of approximately equal length.

Sections 4, 5 and 6 involve considering aspects of Java, and include programming-based exercises. Within these sections you will re-encounter the School System introduced in *Unit 1*. You will not be expected to understand anything more than small (but significant) parts of the code. The School System is used to *illustrate* how particular aspects of Java can contribute towards creating an appropriately structured system: you will be examining code that has already been written, and, on occasions making small amendments to existing code.

Section 4 is a relatively short, straightforward section. One exercise in this section involves the generation and viewing of the Javadoc for a project – if required, more details about generating and accessing project Javadoc in NetBeans can be found in Subsection 3.5 of the *NetBeans Guide*. Sections 5 and 6 are longer sections, involving lengthier practical exercises, and you should expect them to require the most study time of all the sections.

2 Software quality

What are features of good-quality software? As a minimum, software should be delivered on time and on budget (aspects of a successful development process), and should reliably carry out its specified tasks (aspects of a successful 'finished' product). Besides these features, which are perhaps obvious, there are other characteristics of software that are perceived as enhancing quality. In fact, defining and measuring software quality is an active area of research, covering such topics as the following.

- ▶ **Efficiency:** how well does the system carry out its tasks, including making use of the hardware (e.g. the processor) and other software (e.g. the operating system)?
- ▶ **Portability:** how easily can the system run on different platforms, i.e. different hardware and software combinations?
- ▶ **Testability:** how easily can the software be tested?

In this section, we concentrate on two aspects of software quality which generate much debate amongst software developers and which, as you will learn throughout this unit, have a significant impact on the design of software. They are *maintainability* and *reusability*.

2.1 Maintainability

In *Unit 1*, maintenance was introduced as a phase of software development. It covers tasks required to keep the software in use and performing to the satisfaction of its users, after it has been deployed. Such tasks are many and varied. In *Unit 1* we listed the following:

- ▶ fixing emerging problems;
- ▶ fine-tuning the system to improve its performance;
- ▶ enhancing the system by adding extra facilities.

It might initially surprise you that maintenance is considered a *development* phase, but this is sensible because the ultimate goal of software development should be a system that satisfies its users in the long term as well as the short term. It is common for a large-scale software system to have maintenance costs amounting to well over 50% of the total cost of the system over its lifetime. Maintenance, therefore, is an important consideration for developers and for their clients, and it follows that a desirable characteristic of software is that it is easily maintainable – in other words, that it has good **maintainability**.

To further understand why maintenance is such an important activity, we will look in detail at some of the tasks involved.

Fixing problems

No client would be happy with a system that worked perfectly on its first day but failed the next. Of course, strenuous efforts should be taken to ensure that software works as expected through careful development of the software from the start and, not least, robust testing. But testing is not a precise science, and it is commonly accepted even by the most demanding of clients that flaws are likely to exist in large-scale systems, despite testing.

What does maintainability mean in relation to fixing errors? It may seem paradoxical to attempt to create software in which any errors undiscovered by testing are easily fixed: how do developers do this if they do not know where and what the errors are? But as this unit progresses, you will learn about an approach to structuring software which restricts the effects of errors, and which makes errors easier to locate and fix.

Improving performance and adding facilities

In computing, nothing stays the same for long. Operating systems and programming languages evolve continually, and users take it for granted that systems will be updated to take advantage of new features that add more functionality or security, or offer a better user interface, and so on. Software developers ignore the pace of change at their peril, because few clients want a system which is likely to be left floundering at the arrival of the next big software 'thing'. Consider, for example, the two kinds of changes that follow.

► Changing requirements

In *Unit 2*, we discussed the process of specifying the requirements of a system, and we said that it is generally unrealistic to expect requirements to remain unchanged throughout a software development project. The context within which the system is to operate may change during development, for example in commercial environments where the changing operations of the business result in changing system requirements. Also, the stakeholders' ideas of what they want from the system may crystallise only as they see parts of the system developing.

Not unreasonably, what is required of a system can also continue to change after it has been deployed. Additional business needs may emerge; users' increasing familiarity with a system may prompt suggestions for improvements or additional functionality; and so on. Consider a library system designed to deal with online reservations and loans of books. After a period in operation, the library administrators may decide that the system should also enable borrowers to record reviews of books they have read.

► Changing user interfaces

A common impetus for change is the arrival of a new means of user interaction with software. Recent years have seen rapid and exciting developments in user interfaces, which have changed from text-based, command-line interfaces to graphical user interfaces; from handwriting-recognition interfaces to speech-recognition interfaces; and beyond. Systems often have different, or additional, user interfaces 'bolted on' to the same underlying software during their lifetime.

Further, there is a growing number of input and output devices that a system might need to work with. Library systems, for example, make extensive use of special-purpose scanners to read library cards and scan books. These sorts of input device are very likely to change over time.

Whatever the reasons for change, having invested a considerable amount of money in their system's creation, a client is unlikely to countenance its being substantially revised or even completely replaced to accommodate the change. Instead, they will want a system which is sufficiently modifiable that future development is likely to be possible with minimal change to the system.

Legacy systems

A software system can become very difficult to maintain when progress in computing leads to significant differences between the system and the technical environment within which it is operating. For example, a system implemented before the advent of the internet may be written in a language which renders online connections to the system complex and laborious. Furthermore, the company may no longer count amongst its staff people with sufficient expertise in an outdated language. For such reasons, updating the system further becomes problematic.

It may, however, be that the cost of replacing the system is unacceptable compared to the cost of maintaining the system as it is, so the system remains in operation. In such circumstances the system is referred to as a *legacy system*.

The financial services sector retains a large number of legacy systems implemented several decades ago in the language COBOL.

Exercise 1

In this exercise you will consider what might be required to maintain the Hospital System. There are no absolute right answers: the aim is for you to think through some of the issues involved, guided by the kinds of maintenance task you have been introduced to.

The Hospital System's requirements are given in Section 3 of the *Handbook*.

Briefly review the requirements of the Hospital System, and imagine what might happen to the system after it is put into operation. Suggest an example of each of the following:

- (a) an additional requirement that may emerge;
- (b) a change to the Hospital System's user interface.

Discussion.....

Here are our suggestions. You may have thought of different scenarios.

- (a) An additional requirement might be to produce a list of all patients on a given ward who have not been treated by any doctor. You might have thought of something further removed from the current requirements, for example, a requirement for the hospital system to have online access to patient records at local health centres.
- (b) In future, a patient's details might be entered by scanning his or her identity card.

2.2 Reusability

From the earliest days of computer programming, people found themselves repeatedly addressing the same computing tasks. Often what they wrote was virtually identical to what they or someone else had written many times before.

How could this wasteful reinvention be avoided? The first stage was to recycle ways of doing things: reusing *ideas*. For example, if someone wanted a method to sort a list of items into numerical or alphabetical order, they could simply look at how others had designed their software to carry out the same task.

In fact, in many situations entire portions of code could be reused. If two programs needed to carry out the same task, the relevant part could simply be copied from one program to the other. This approach can work, but it has a serious weakness. Imagine you have copied a chunk of code to many other working programs and then you realise that there is a much better way of writing the code. Or perhaps you discover that the

original code does not work in all circumstances and needs correcting. Now you have to find all the places the code has been reused and bring them up to date. This will take a great deal of effort, and some copies will probably be missed.

A far better idea is to write the piece of code just once and have all the programs that need to use it refer to it in some way. Then only this shared copy will need updating. In this way the code itself is reused.

SAQ 1

Identify a way in which you have reused existing code in your own programming work.

ANSWER.....

There are many ways in which you might have reused code, though you may not have considered what you were doing as reuse.

For example, whenever you use predefined Java classes (think of `HashSet`, `FileOutputStream` and so on) you are reusing code that someone else wrote. Even creating a simple `Account` class with a `name` instance variable referencing a `String` object is an example of reusing the `String` class.

Good reusability is a highly desirable feature of software. It is an umbrella term covering many aspects of avoiding unnecessary duplication of effort. Software deemed to exhibit good reusability may have been developed using existing artefacts (designs or code, for example), or it may itself offer artefacts for future reuse, or both.

The Iridium satellite system

The Iridium system, a satellite-based communication system enabling wireless communication around the world, was described in the Introduction to *Unit 1* as an example of a very large software development project.

More than 15 million lines of code were required in this project. Of these, over 85% were reused from previous programming projects – a huge saving in time and resources.

In this section you have learnt that maintainability and reusability are key influences on software design and implementation. In the remainder of this unit you will learn how software can be developed to enhance these qualities.

3

Software components

Section 2 focused on some important properties that software should possess, in particular *maintainability* and *reusability*. Using software components to develop software is one of the keys to enhancing these qualities, and in this section you will learn:

- ▶ what is meant by a software component;
- ▶ what is meant by a component-based approach to developing software;
- ▶ about the characteristics of object-oriented software components.

3.1 What is a software component?

To set the scene for this subsection, consider the production of cars. Throughout the history of the automotive industry, cars have been built from components – wheels, gearboxes and so on. At first, product design (that is, car design) came before component design: the components were made to suit the product, and could not be used for other products, which was very inefficient.

Nowadays, however, most of the component parts exist before the end product, which is designed on the assumption that as far as possible it will be built from standard and interchangeable parts. This means that the cost of designing the parts in the first place is spread over all the products that use it, so the end product can be cheaper. If a part is in standard use, any faults will already have been found and corrected, so the end product will be more reliable. Parts are interchangeable, which means that the customer can be offered a range of options, and any maintenance or upgrading required should be much simpler. And because the assembling of the parts is well defined and well understood, the production process is cheaper and quicker. In summary, building cars in this way leads to products that incorporate desirable qualities such as reusability, affordability, reliability and maintainability.

In the computer industry, although computer *hardware* is designed and constructed in a similar fashion to cars, *software* is predominantly individually crafted, with bespoke software (i.e. software specially built for a particular use) being the norm. In addition, while the cost of reliable computer hardware plummets, software is often costly, inflexible and unreliable. Frequently, software projects are delayed, run over budget or produce software of unacceptable quality.

Problem projects?

In 2004, Computer Weekly magazine and Templeton College, Oxford, published the results of a survey of 1500 software project managers. This revealed that:

- ▶ the number of projects completely achieving their targets was 16%;
- ▶ the average overspend on a project was 18%, with an average excess development time of 23%;
- ▶ the average underachievement in software functionality was 7%.

Sauer and Cuthbertson (2004)

Many software developers believe that the key to creating software which avoids these problems, and which embodies desirable software qualities, is to find a way of imitating hardware production by using existing software components that can be systematically assembled to achieve the required product. That is, following a **component-based approach** to development and aiming towards **component-based software**. The goal of component-based software development is not, however, the mass production of software assembled on production lines. Rather, it is that software components be employed to assemble reliable software quickly and inexpensively, whether creating a one-off program for a particular client or creating a software product for the mass market.

Consequently, a **software component** is a piece of software with a well-defined purpose that can be combined with other pieces of software to construct a larger system. For example, an `AddressManager` component might offer facilities concerned with the storage and manipulation of names and addresses, such as finding the addresses associated with a given name, or listing all addresses in a certain town. It could be used in a variety of systems: a library administration system, a company's customer records system and so on.

SAQ 2

Consider a word-processing application with which you are familiar. Suggest two components that might have been used in its creation.

ANSWER.....

There are many possibilities for software with a well-defined purpose that could have been used. Two that we thought of are a drawing component and a spellchecker.

In the rest of this section, we will assess the extent to which the advent of object-oriented technology has made component-based construction of software a practical proposition, by comparing the characteristics of components used by engineers with those of object-oriented software. Because engineers design and construct tangible artefacts, it is convenient to refer to them as **engineering components**. We start by considering engineering components in some detail.

The term 'engineering components' should be taken to encompass what in computing circles are termed hardware components.

3.2 Engineering components

Engineers do not specify a design where every component has to be manufactured specially, but base their design on components that have been tried and tested in other systems. These may include small components such as nuts and bolts, as well as more complex items such as engines and fuel pumps. Importantly, when new components have to be designed, it is with the expectation of reusing them later in other products. In other words, engineers:

- ▶ use existing components;
- ▶ design new components so that they can be reused.

The design and construction of products is aided by certain characteristics of engineering components, which we will illustrate by considering the components that constitute a particular product: a domestic hot-water and central-heating system, as in Figure 1.

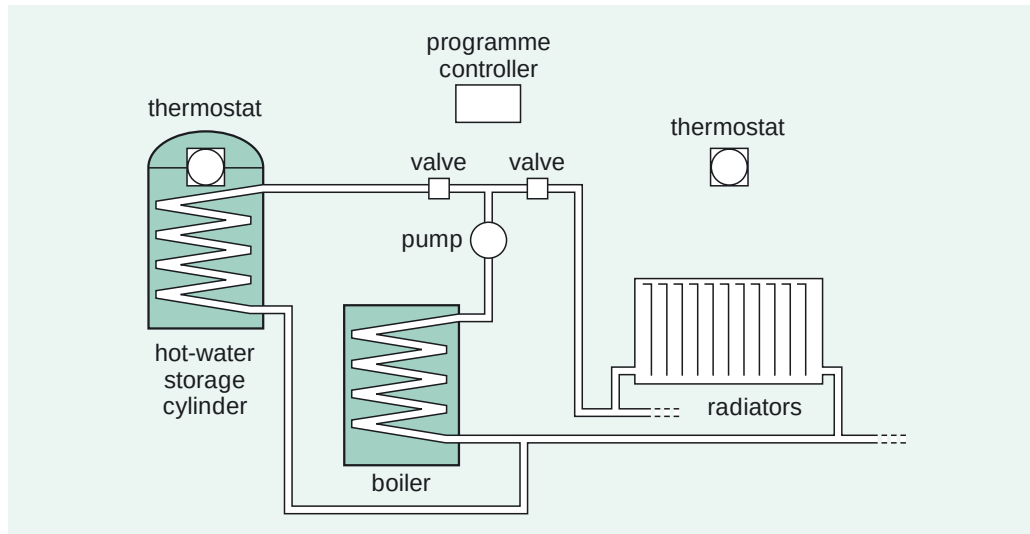


Figure 1 The components of a domestic hot-water and central-heating system

Engineering components as independent building blocks

Engineering components are designed and packaged as interacting yet independent building blocks that may be used in the construction of different products. For example, the components used to construct a domestic hot-water and central-heating system are interconnected and interact with each other. The electric water pump receives hot water supplied from the boiler. It distributes water through the valves to the storage cylinder and the radiators.

The storage cylinder and the radiators rely on the services provided by the pump, but they are independent of *how* the pump works. The pump could be replaced by one which offers the same services but whose construction is different – it might be more efficient – without affecting the rest of the system.

Engineering components as a hierarchy of building blocks

Each engineering component itself may be constructed from several simpler components, which themselves may be constructed from several yet simpler components, and so on until the components are fundamental building blocks. That is, components are constructed from a hierarchy of building blocks where the smallest elements, or 'leaves', are fundamental building blocks.

For example, an electric water pump is constructed from two components – a water pump driven by an electric motor. The electric motor is comprised of an armature and an electromagnet, both of which are constructed from coils of insulated copper wire.

Specification of an engineering component

An engineering component is designed and built to a **specification**, which describes:

- ▶ its interface (how it connects to other components);
- ▶ its requirements (what it needs);
- ▶ its services (the results of correct usage).

Standards for specifications may be agreed within a particular industry, or may be the subject of national or international agreements. Conformance of specifications to standards facilitates the design and construction of products from components that may be manufactured by different companies.

In the heating system, the electric water pump is connected into the system by copper piping. Its pipe connections are designed to accept the standard diameters of

manufactured copper piping. The pump requires a specified standard electrical power supply, and it might have other specified requirements such as a limit on the surrounding temperature. If its requirements are met, then it will provide specified services such as pumping water at a certain rate.

A component may be replaced by another one that meets the same specification, conforming to the same standards and working under the same conditions to give the same or improved services.

SAQ 3

How do standards facilitate

- (a) the construction of products and
- (b) the maintenance of products?

ANSWER.....

- (a) Components can be combined with other standard components to construct new components or products.
 - (b) When a product needs to be adapted or upgraded, constituent components are readily replaceable by components that are produced to the same standards.
-

Different perspectives on an engineering component

It is instructive to compare the perspectives of engineering components held by the people who design and manufacture them with those held by the people who use them to construct a product.

A manufacturer designs a component to meet the specification for that component. The manufacturer will be fully conversant with the underlying technology required to implement the component, and also with any relevant standards.

The users of a manufactured component will employ it to meet the requirements of the product that they are building. To the users, the component is a 'black box'. It should not be necessary for them to be conversant with the underlying technology used to manufacture the component. They only need to know how and when a particular component can be incorporated into a particular product, so they will need to be conversant with the specification of each component they use.

For example, when selecting a suitable electric water pump for the heating system, a plumber may be able to choose from a number of alternatives, all satisfying certain standards. The plumber does not need to be knowledgeable about the underlying technology required to implement a pump. A particular pump will be selected because its specification meets the requirements of the particular system that the plumber is constructing.

SAQ 4

What do the different perspectives of a component held by its manufacturer and its users have in common?

ANSWER.....

The different perspectives are united by the specification of the component. The manufacturer produces the component to a specification. The user employs that component because its specification meets the requirements for a component that is needed to build a product.

The characteristics of engineering components that we have discussed can be summarised as follows.

- ▶ Components are designed and packaged as interacting yet independent building blocks that may be used in the construction of different products.
- ▶ Components are constructed from a hierarchy of building blocks which are themselves components.
- ▶ Components are designed and built to a specification.
- ▶ Manufacturers and users of components have different perspectives on the components, but have the specification of the component as a common element.

3.3 Object-oriented software components

To assess the extent to which object-oriented technology has made component-based construction of software a practical proposition, we now consider whether objects, as the smallest object-oriented elements, share the characteristics of engineering components described above.

Of course, an object does not exist in isolation, but is an instance of a particular class which defines its behaviour. Consequently, depending on the context, we will refer to either classes or objects as components.

In practice, to be a useful reusable entity, a **component** may be larger than a single class. It is often not sensible or possible to use a class in isolation: the class may only make sense in the context of its objects' interactions with objects of another class, or within an inheritance hierarchy of classes, for example. Different programming languages have different facilities for combining classes into a single component; in general, though, the properties of a component are largely determined by the properties of its individual classes, so we initially concentrate on individual classes. Whilst we use Java by way of illustration, we concentrate on features which are common across object-oriented languages. We consider some Java-specific issues later.

Classes as independent building blocks

Recall that engineering components are:

- ▶ designed and packaged as building blocks;
- ▶ interacting;
- ▶ independent.

In the discussion that follows, you will see that classes/objects have all these qualities.

Building blocks

A fundamental concept in object-oriented technology is **encapsulation** – the packaging together of data and the operations that can be applied to that data. This means that everything needed for the working of a class of objects is contained within the class. The definition of a class includes the following.

- ▶ A **specification** of each method that can be invoked on objects of that class; that is, a description of its purpose and usage. Such method specifications, together with specifications of inherited methods, constitute the class's **protocol**, defining the corresponding messages that its objects will respond to.
- ▶ The class's **implementation details**; that is, how it is coded, including the code for its methods and constructors, and the declaration of its instance variables.

Thus, everything needed to implement and interact with its objects is defined by the class, providing a building block that can potentially be reused in different systems requiring the services offered by objects of that class.

Interaction

Object-oriented software is based on the notion of a collection of objects that interact with each other to perform some useful purpose. This interaction is effected by objects communicating with each other via messages, that is, collaborating with each other. In a collaboration, one object (the *client*) requests a service of another object (the *server*).

Each object has a well-defined set of **responsibilities** – tasks, or services, which it carries out on receipt of appropriate messages. Taken together, an object's responsibilities define its **behaviour**.

Independence

A key concept in object-oriented technology is that of **data hiding**. Data hiding means protecting an object's implementation details by restricting access to them. An object's clients should interact with it through sending messages corresponding to the protocol offered by the object, rather than by directly accessing its code or state. Data hiding provides a basis for the *independence* of classes. Clients of an object should be unaffected by changes made to the implementation of the object's class, provided its protocol remains unchanged.

Data hiding also protects the object's **integrity**, that is, the validity of its state. For an object to be in a valid state, the values of its instance variables must be consistent with any invariant on the object's state. (For example, the age instance variable of valid *Pupil* objects may need to be constrained to range from 4 to 18.) If the only way of interacting with an object is through sending it messages, then the possible effects of an interaction on the object's state can be controlled by careful coding of the methods of its class.

The concept of data hiding is illustrated in Figure 2. The extent to which it is enforced varies between languages: we will expand upon Java facilities for data hiding later in this unit.

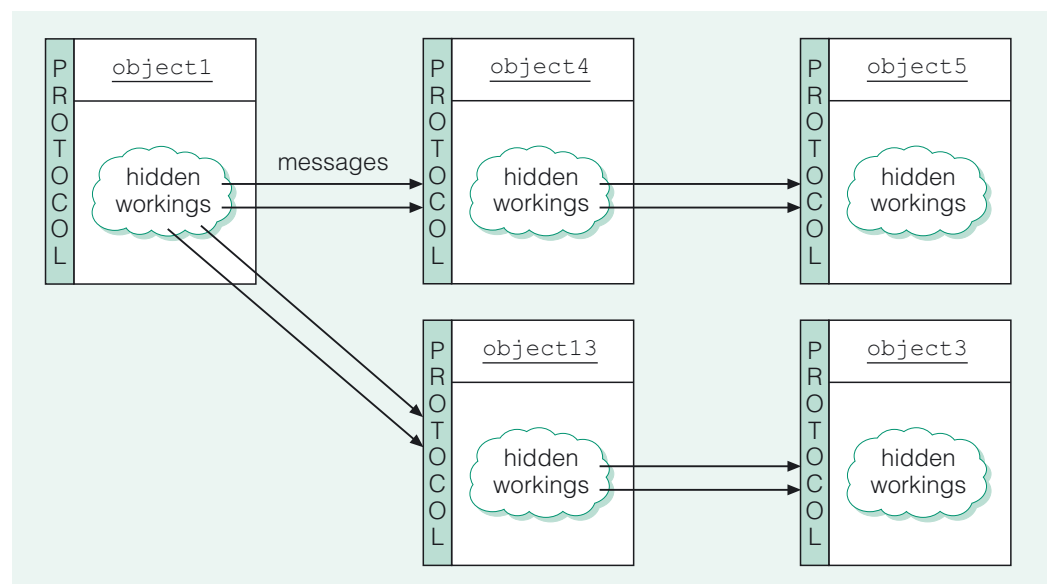


Figure 2 Data hiding

It should be said that clients do not usually deliberately invalidate an object. Indeed, it is usually in their best interests not to do so, because the consequences can be unpredictable and far-reaching. But the developer of a class needs at least to guard against accidental misuse.

So it is clear from the discussion above that, akin to an engineering component designed for some tangible product, an object is an interacting, independent building block for a software system.

A hierarchy of building blocks for classes

In the case of objects, the state of an object is made up of other data in the system, which the object references. In Java, this 'other data' may consist of objects or values of primitive types. Each object so referenced may itself reference other data, and so on until the data items involved (the 'leaves') are all values of primitive types such as numbers and characters, that is, fundamental building blocks that cannot be further broken down. The state of each object can therefore be viewed as a tree-like structure constructed from a hierarchy of objects and primitive data values, as in the following object diagram.

We are not discussing subclasses and superclasses here, although they also form hierarchies.

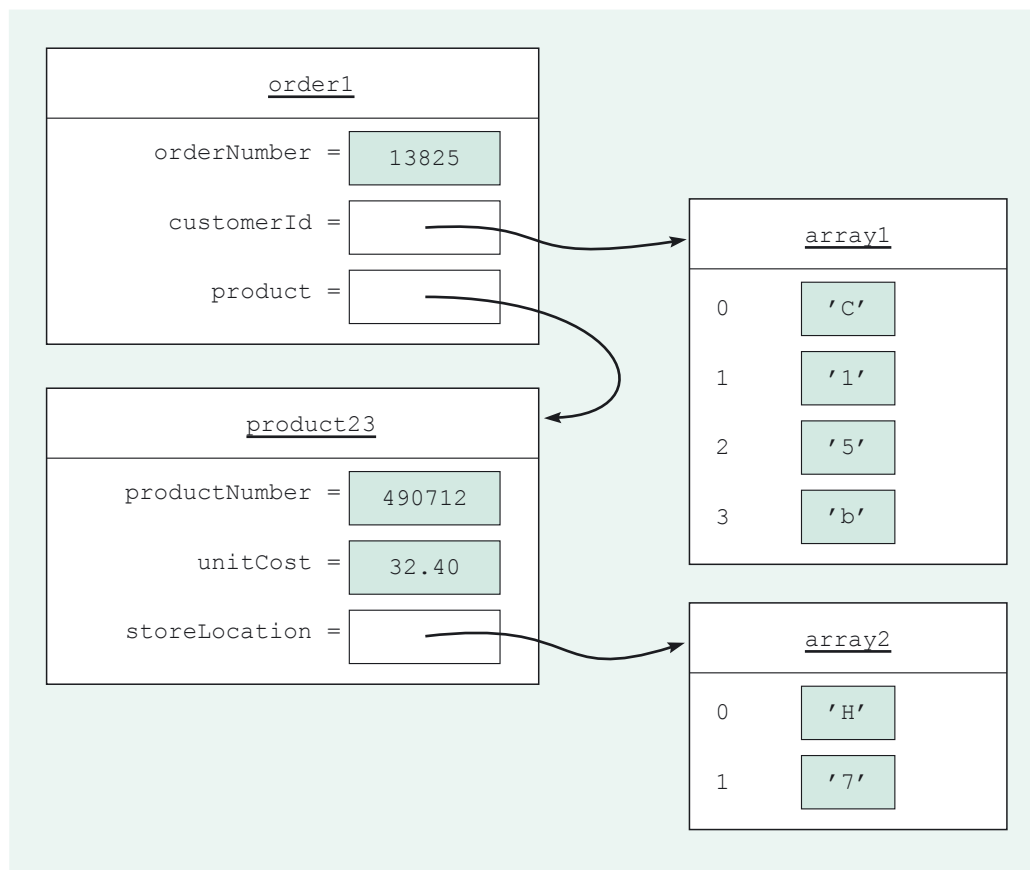


Figure 3 A tree-like hierarchy, illustrating the state of an object, `order1`, of a class `Order`. Arrows denote references; 'leaves' are highlighted

Exercise 2

An instance, `account1`, of a Java class `Account` has instance variables as follows:

balance, with value 555.25 (double);
 interestPaid, with value true (boolean);
 passwords, referencing the `HashSet` object {"\$abc", "\$xy"}.

- Draw a tree-like hierarchy of the objects and primitive data values that make up the state of `account1`.
- What are the 'leaves' in this case?

You should assume that the state of a `HashSet` object is made up of the elements it contains, and that the state of a `String` object is made up of its individual characters.

Discussion.....

(a) Figure 4 shows the hierarchy of objects and data values.

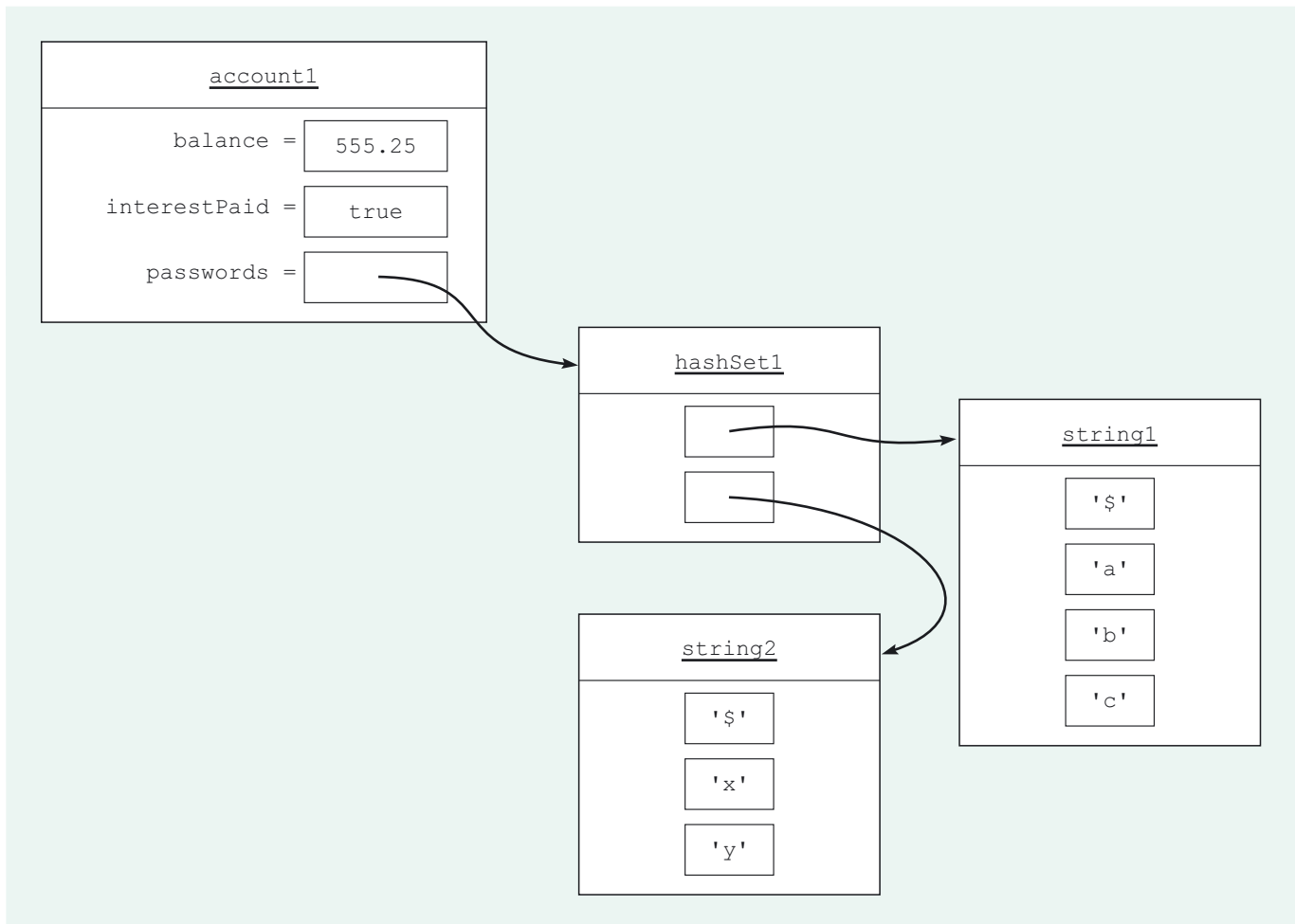


Figure 4 Object diagram showing an object of the class Account

(b) The 'leaves' are:

- ▶ the double value 555.25;
- ▶ the boolean value true;
- ▶ the char values '\$', 'a', 'b', 'c', 'x' and 'y'.

In summary, objects, like engineering components, are constructed from a hierarchy of building blocks.

Specification of a class

We stated that an engineering component's specification describes:

- ▶ its interface (how it is interacted with);
- ▶ its requirements (what it needs);
- ▶ its services (what it does).

Exercise 3

What is required to specify a class in a similar way to an engineering component?

Discussion.....

Information must be given about:

- ▶ the class's protocol (which defines how to interact with objects of the class);
 - ▶ any other classes it requires;
 - ▶ what using the protocol achieves.
-

As you will see in the next section, a Java class specification can be automatically generated from the class definition using the Javadoc program.

Because engineering components are built to a specification, one component may be replaced by another one meeting the same specification. In object-oriented software, the concept of **polymorphism** is required to achieve the analogous effect with classes. Polymorphism is the capability for objects of different classes to respond to the same message in a manner appropriate for each object.

Polymorphism is sometimes described as meaning that a message can be sent without concern for the class of object receiving the message: provided that object has the message in its protocol, the behaviour appropriate for that object will result. This is clearly needed to freely replace one class by another in a system. The extent to which simply replacing one class by another is actually feasible depends on the details of how polymorphism works in the language concerned; we will not be investigating this issue further.

Like engineering components, then, classes are built to a specification. However, the concept of standards for software components is far less advanced than that for engineering components. We return briefly to this issue in Section 7.

Different perspectives on classes

We noted above that the definition of a class involves two distinct parts: the specification of its protocol; and its implementation, which ensures that objects of the class can respond to its protocol. A client using a class need only know about its specification – what it does and how to use it – whereas the class's designer must be concerned with its implementation. In other words, as in the case of engineering components, *users* of a class have a different perspective from that of its developers.

Engineering components compared with software components: a summary

From this comparison of the characteristics of engineering components with those of classes (or objects) as software components, it can be concluded that there are sufficient similarities that object-oriented technology could facilitate component-based software development, with object-oriented features such as encapsulation, protocols, data hiding and polymorphism contributing to this capability.

Components are not unique to object-oriented software. Software implemented in other kinds of language may involve the notion of a component, manifested in modules, library files, functions and so on. But the fact that the most elemental structure in object-oriented software – an object – has component-like characteristics means that object-oriented technology lends itself to components more naturally and successfully than other approaches.

3.4 Larger software components

We noted previously that a class may require the presence of other classes in order to function. This dependence between classes should not be surprising, because the interaction of objects to perform tasks is a fundamental object-oriented idea. There are several ways in which a class `ClassA` can be dependent on another class `ClassB`. Here are some.

- 1 `ClassA` may be a subclass of `ClassB`.
- 2 `ClassA` objects may require a service provided by `ClassB` objects in carrying out a task. That is, inside one of `ClassA`'s methods, a message is sent to a `ClassB` object.
- 3 `ClassA` may have a method that receives or returns a `ClassB` object.
- 4 `ClassA` may define an instance variable referencing a `ClassB` object.

A class that depends on other user-defined classes, that is, those not provided by the language environment, will only be reusable if the services provided by those classes are available in any new system. This may mean that the classes involved should all be part of the same component. Even if classes are independent, they may offer related services, and so be combined to form a sensible unit of reuse.

Consideration of the need to combine dependent and related classes leads to the following description. An **object-oriented software component** consists of one or more classes grouped together in some way to form a piece of software that has a well-defined purpose and can be combined with other pieces of software to construct a larger system. Interacting with the component means, in effect, interacting with one or more of the classes within it; the component's **protocol** is taken to mean the combined protocols of its constituent classes.

An object-oriented software component should have a **specification** which includes:

- ▶ a description of how it should be interacted with (its protocol);
- ▶ a description of any other components it requires;
- ▶ a description of what results from using it in the specified way.

It can be helpful to think in terms of a contract. If the services needed by the component are provided, and if it is interacted with in the specified way, then it will guarantee its services as described.

In this section you have seen that in object-oriented software development, classes/objects are 'natural', albeit small, software components, which are often grouped together to form larger components. In the next section you will review how components can be formed in Java.

The grouping mechanism varies between languages.

The term *interface* is sometimes used in the context of a component to mean the same as protocol. We will not use *interface* in this way, to avoid confusion with the Java *interface* construct.

In the rest of this course, unless indicated to the contrary, we will use the terms 'component' and 'software component' to mean 'object-oriented software component'.

4

Java packages and specifications

In the previous section you learnt that in object-oriented programming a class, or group of classes, can be considered as a software component.

In this section, we will describe how Java packages can be used to group one or more classes into larger components, and how packages are:

- ▶ created and used;
- ▶ documented;
- ▶ distributed.

4.1 Working with packages

It can be useful to group Java classes together in a Java **package**, and consider that package as a component. Any set of classes can be grouped together in this way, but if packaging is used sensibly on a large project, then a package will normally consist of dependent or related classes which combine to offer a coherent set of services.

When you define a class, you can specify that it will belong to a particular package by giving the name of the package. For example:

```
package mypackage;
```

Creating a new package is as simple as declaring its name in this way at the beginning of a class definition. Alternatively, you may choose not to specify any package. In that case, Java will put your class in the **unnamed package**, which contains all the classes which are not in any named package.

In fact, Java is fundamentally premised on the concept of components structured into packages. Its **application programming interface (API)** supplements the core Java language and makes available numerous components in the form of reusable packages. You will have met many of these predefined 'libraries': programming without them is very limited. They offer sets of useful services such as those associated with collections (e.g. maps and lists), input and output streams, and graphical elements such as buttons and text boxes. When you program with Java, you use its API by importing classes from predefined packages into your source code. For example, including in a file the code

```
import java.io.FileInputStream;
```

makes available to that file the class `FileInputStream`, which is contained in the package `java.io`.

The following code makes available all of the classes contained in the package `java.io`:

```
import java.io.*;
```

Having either of these import statements in a file allows you to use code such as:

```
FileInputStream fis = new FileInputStream("School.data");
```

Although the statement

```
import java.io.*;
```

makes available the *classes* in the package `java.io`, it is often stated that it imports the *package* `java.io`. However it is important to realise that it is not actually the package that is imported but only the classes that are in it. This is why when 'a package is imported' the classes in any sub-packages that it contains are *not* imported.

By convention, package names are lowercase throughout.

More generally, the services offered by a component may be referred to as the component's API.

The package `java.lang` is automatically available to you. It contains classes fundamental to Java programming, such as `String`.

The Java interfaces defined in a package are also made available by such an `import` statement. We do not discuss Java interfaces in this unit.

It is possible to make use of a predefined Java class without using an import statement, by using its fully qualified class name. For example the following is an alternative to the previous two lines of code:

```
java.io.FileInputStream fis = new java.io.FileInputStream("School.data");
```

A user-defined package is imported in just the same way as a predefined package. For example:

```
import mypackage.*;
```

4.2 Documenting packages

When you program with Java, the implementation details of its API are not normally readily available to you. For each package, you work from a specification – supplied documentation which describes the purpose and usage of the package, giving a specification for each class that is available to a client of the package. Figure 5 shows an excerpt from the documentation for the class `ArrayList` which is provided in the package `java.util`. It is the specification for the method `add(int index, E element)`.

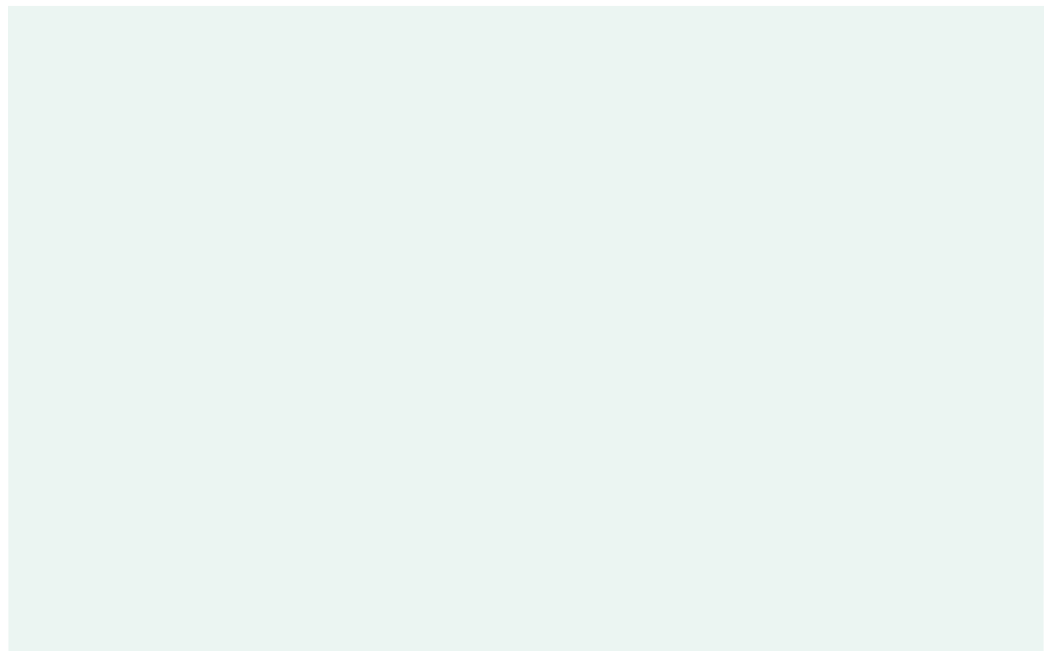


Figure 5 Excerpt from the API documentation for the class `ArrayList`

SAQ 5

The above specification includes no detail of how the method is implemented. There is no information about the code used to move the list elements, for example. Why is this?

ANSWER.....

The specification includes everything a user needs to know in order to use the method appropriately. It is assumed that the user should not need to know anything about its implementation.

A method specification should make clear the circumstances under which the method may be invoked. That is, it should define the method's **pre-conditions**. A method need not have any pre-conditions specified.

SAQ 6

From the above specification, identify a pre-condition that is a restriction on the range of an index supplied as argument to the `add(index, element)` method.

ANSWER.....

The index should be between 0 and the size of the receiver, inclusive. This is evident from the fact that an exception is thrown otherwise. This restriction is a pre-condition on this method.

Recall that exceptions are Java's particular way of handling various kinds of error situation.

A method specification should also set out what invoking the method achieves, assuming the pre-conditions are satisfied. That is, it should define the method's **post-conditions**.

SAQ 7

From the specification shown in Figure 5, identify the post-conditions for the `add(index, element)` method.

ANSWER.....

The post-conditions are described in the statement beginning 'Inserts the specified element ...', which appears after the method header `public void add(int index, E element)`. That is, the post-conditions are that the specified element is situated at the specified position in the list, with any element already at that position, and any subsequent elements, shifted to the right.

When you create your own packages, the Javadoc program provided with the Java SDK, and also as part of NetBeans, can be used to produce a specification of your component for other users. As you saw in *Unit 1*, Subsection 6.2, Javadoc extracts doc comments, delineated by `/**` and `*/`, and uses them to provide information about each class for its clients. The Javadoc documentation is produced as an HTML file, which can be displayed in a browser window, containing information such as that in Figure 5.

In the following exercise, you will review the package structure and some of the documentation for the School System which you met in *Unit 1*.

Exercise 4



In NetBeans, open the project `School`, which is in the folder `M256\M256Code\System`.

- The `School` source code is structured into two parts, called `schoolcore` and `schoolgui`. These are in fact the two packages, developed by the M256 course team, that constitute the School System. Briefly look at the classes each package contains.
- Now turn to the definition of the class `SchoolCoord` in the file `SchoolCoord.java`, which is in the `schoolcore` package.

Explain the effect of each of the following lines of code from the class definition (with line numbers added for reference).

```
package schoolcore;    //1
import java.util.*;    //2
import java.io.*;      //3
import m256date.*;     //4
```

- Read the doc comments in the code for the `SchoolCoord` method `enrolPupil(aName, aBirthDate, aForm)`.

You might like first to refresh your memory of NetBeans by rereading Subsection 3.1 in the NetBeans Guide.

To generate the Javadoc, from the Build menu choose Generate Javadoc for "School", or alternatively right-click on School in the Projects window and select Generate Javadoc for Project.

Remember the term *form* is here being used as a synonym for *class* (of pupils).

If a browser window does *not* open automatically, the Javadoc can be accessed by selecting Window|Files. Then, in the Files window, expand the School node. Next, expand the newly revealed dist node, and from there expand the javadoc node. Right-click on the index.html file and select View.

Generate the Javadoc documentation for the `School` project and read it. What are the pre-conditions for the above method? What are its post-conditions?

Discussion.....

- (a) The `schoolgui` package contains a single class, `SchoolGUI`. The system's graphical user interface is implemented as an object of this class.
- The `schoolcore` package contains the classes `Form`, `Teacher` and `Pupil`, which correspond to entities within the real-world environment of a school. It also contains `SchoolCoord`, an instance of which, as we discuss further later, has a particular role in communicating with the user interface.
- (b) Line 1 declares that the class being defined – `SchoolCoord` – is situated within the package `schoolcore`.
- Lines 2 and 3 respectively make available the classes (and interfaces) in the predefined packages `java.util` and `java.io`. As each class is defined in a separate file, if the code for another class in `schoolcore` requires the use of `java.util` or `java.io`, the required package must be separately imported into that class's file.
- Line 4 makes available the classes in the package `m256date`. This is a package created by the M256 course team, containing, in fact, just the class `M256Date`, which simplifies your work with dates.
- (c) The doc comments appear before the method header, starting with `/**` and ending with `*/`.

On generating the Javadoc documentation, the Java documentation should appear in a browser window.

The method's pre-conditions are:

- that the pupil should be aged between 4 and 18 inclusive;
- that the form should have fewer than 10 pupils in it.

The method's post-conditions are that a new pupil is created with the given name and birth date and added into the form.

You have seen that in the School System, user interface code is packaged separately from the rest of the system code. Figure 6 gives an illustration.

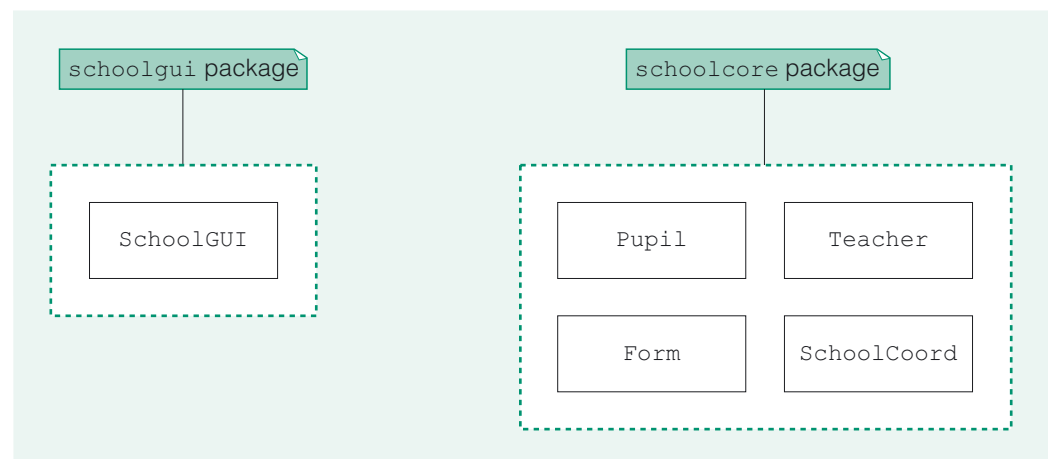


Figure 6 The School System packages

This follows a key design principle to which we will refer frequently in this course: the separation of the *core system* (a term for the underlying software introduced in *Unit 1*) from the mechanism that allows the user to interact with the core system.

There are several advantages to this arrangement, notably that the user interface can be independent of how the core system is implemented, so that it does not have to change when the core system implementation does. Similarly, it should be possible to change and even replace the user interface without impact on the core system.

4.3 Distributing packages

In carrying out the last exercise, you had access to the source code for the packages `schoolcore` and `schoolgui`. Typically, a component developer does not make their source code readily available to users, providing them instead with the compiled code (`.class` files) and the specification, that is, with everything required to use the software but not to see its implementation. This is what the M256 course team did with the package `m256date`, providing it in the form of a **JAR file**. This is a form of zip file used to distribute compiled code, documentation and other information required for the software to be used. (NetBeans can produce JAR files automatically: M256 does not require you to do this.)

There are tools called **decompilers** that take compiled Java files and derive the source code. So it is possible, even with the predefined Java API, to find out about the implementation of components. The point is not that a component user should be prevented from seeing the implementation, but that the component user should rely only on what is documented in the component specification. The component's implementation may change, so long as the specification remains the same. If the user has discovered, and relied on, implementation details which subsequently change, then they have only themselves to blame!

In this section you have reviewed the formation, use, documentation and distribution of packages, Java's component-structuring mechanism. Next you will learn about the contribution of data hiding to forming appropriate Java components.

During installation of the M256 software, the `m256date` JAR file was installed as a library in NetBeans so that it is automatically available to you.

5

Data hiding: access modifiers

Section 3 identified data hiding as a key factor in the viability of component-based object-oriented software.

SAQ 8

What is meant by data hiding?

ANSWER.....

Data hiding means isolating an object's implementation details – its code and state – from the object's clients.

You have seen that data hiding is important for two main reasons:

- 1 so that clients of an object remain unaffected by changes to the object's implementation details;
- 2 so that the integrity of an object may be preserved during its use by a client.

SAQ 9

What is meant by the integrity of an object?

ANSWER.....

Integrity refers to the validity of an object's state – whether it is consistent with any invariants on the object and hence whether it conforms to relevant real-world constraints.

The concept of data hiding is common throughout object-oriented technology, though there is variation in how strictly languages enforce it. The aim of this and the next section is to consider aspects of data hiding in Java. The Java features discussed should not be new to you. What may be new is viewing them not just as technical details that have to be applied correctly in order to get programs working (though this is important), but also as tools for creating component-based Java software.

5.1 Data hiding and classes

Java allows the programmer some control over the accessibility to clients of **class members** – the elements of the class definition, such as instance variables, methods and constructors – through **access modifiers**. Access modifiers control the degree to which class members are accessible from outside the class. Before discussing this further, it will be useful to review briefly what 'accessible from outside' means.

Accessing an instance variable from outside its class

Accessing an *instance variable* from outside its class means setting and/or retrieving the value of the instance variable by using the instance variable directly in code external to the class definition. Consider the example of an `Account` class intended to model bank accounts, and which defines instance variables `overdraftLimit` and `balance`.

The code in Figure 7 defines, outside the `Account` class definition, another class `AccountClient`. It illustrates the `balance` instance variable of an `Account` object, `myAccount`, being accessed from outside the `Account` class definition.

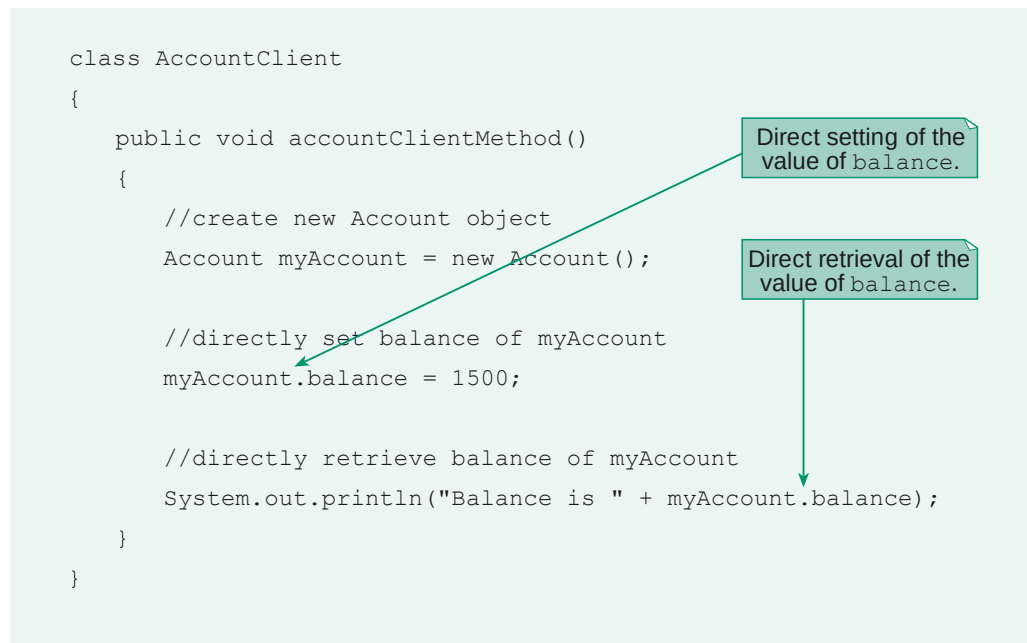


Figure 7 The code for a class which is a client of `Account`

Accessing a method from outside its class

Accessing a *method* from outside its class means invoking the method directly in code external to the class definition. If the `Account` class defined a method `setBalance(anAmount)`, then the following code, if it appeared within the class `AccountClient` above, would demonstrate accessing this method from outside the `Account` class:

```
myAccount.setBalance(300);
```

In this example, the `Account` object `myAccount` is sent the message `setBalance(300)`, which causes the corresponding `Account` method to be invoked.

Access modifiers

Access modifiers are the keywords `public`, `private` and `protected`, which are used to prefix class members in the class definition. They control access as follows.

- `public` Any class, in any package, has access to a public member.
- `protected` The class itself, all classes in the same package as that class, and all subclasses of the class (including those outside the package) have access to a protected member.
- `private` Only the class itself has access to a private member.

In the absence of one of the above modifiers, what is referred to as the *default access modifier* applies, which means that:

the class itself and all classes in the same package as that class have access to the member.

In other words, the default in the absence of an explicit modifier in front of a class member is that the member cannot be accessed from outside the class's package. We will also refer to this as *package accessibility*.

This assumes that the class containing the member has been imported if it is in a different package.

A subclass defined in a different package would *not* have access to the member.

In the M256 systems, we will use public, private and package accessibility to ensure appropriate data hiding. Protected accessibility is more complex and less frequently used than the others, and our systems will not require it.

The following exercise illustrates the importance of using access modifiers to hide data appropriately.

Exercise 5

Earlier in this section we introduced an `Account` class, with instance variables `overdraftLimit` and `balance`, whose instances model bank accounts. There are constraints on bank accounts which should be reflected by the objects that model them, for example:

- ▶ that the overdraft limit of an account cannot be a negative amount;
- ▶ that the balance of an account cannot be such that the customer has exceeded their overdraft limit.

These constraints are mirrored by invariants on the permissible values of an `Account` object's instance variables. For example, if a particular object has `overdraftLimit` with value 500, then setting its `balance` to -600 puts that object into an invalid state. Such an action violates the integrity of the object, and should not be allowed to happen.

Suppose a programmer provides the following definition for the `Account` class. Explain why this code fails to protect the integrity of an `Account` object and how you might remedy this.

```
class Account
{
    public double balance;
    public double overdraftLimit;

    public void setBalance(double anAmount)
    {
        if (anAmount >= -overdraftLimit)
        {
            balance = anAmount;
        }
    }
    //code omitted
}
```

Discussion.....

The class definition does not guarantee the invariants on an `Account` object's state.

The method `setBalance(anAmount)` does correctly check a proposed new balance value against the overdraft limit before setting the balance. And it is available to clients using `setBalance(anAmount)` messages, since the method has public accessibility.

But there is another way for clients to set the balance of an `Account` object. The `balance` instance variable is exposed to the world, as it has public accessibility. This enables any client to access `balance`, and set it directly without restriction, thus potentially violating the `Account` object's integrity.

Similarly, the fact that `overdraftLimit` has public accessibility means that the overdraft limit of an `Account` object could be set by a client to a negative amount, or to a value inconsistent with the balance.

Irrelevant code, and all comments, have been omitted.

To fix this, the `balance` and `overdraftLimit` access modifiers should be changed. Declaring these instance variables as `private` would mean (in the absence of any other `Account` methods modifying these instance variables) that a client outside the `Account` class could not change an `Account` object's overdraft limit, and could only change the balance of an `Account` object using the method `setBalance(anAmount)`. The integrity of `Account` objects would be safeguarded.

Common advice given to novice Java programmers is to give instance variables `private` accessibility and methods `public` accessibility. The consequences for an object of a class defined according to this policy are usually described as follows.

- ▶ A client outside the class is not privy to the object's internal implementation details, which might change.
- ▶ The state of the object cannot be directly accessed, and so potentially invalidated, from outside the class. The object's integrity is protected.
- ▶ A client outside the class can interact with the object by invoking its public methods, through sending the corresponding messages.

The policy appears to guarantee data hiding. Of course, the protection of an object's integrity relies on the programmer only providing methods which maintain integrity. Had the following method been provided in `Account` then, even if `balance` had `private` accessibility, the threat to an `Account` object's integrity would remain, because the method does not check the balance against the overdraft limit.

```
public void setBalanceUnrestricted(double anAmount)
{
    balance = anAmount;
}
```

Using only `public` and `private` in this way is a reasonable initial policy which generally serves well in the creation of small programs. With more complex systems, you may need to make further use of the flexibility offered by the access modifiers. Constructing a *package* appropriately requires judicious use of the default access modifier (i.e. package accessibility), as well as `public` and `private`, as we now explain.

You will learn in the next section about certain circumstances in which an object's integrity can still be compromised.

5.2 Data hiding and packages

Because the creation of a package is often under the control of a single developer or team of developers, protecting the objects within a package from each other is often considered less important than protecting the package as a whole from inappropriate external use. Objects that are packaged together may allow each other a level of access to their internal details and to their services beyond that which they offer to a client of the package. Package access can be used to achieve this. Figure 8 shows an illustration of object interactions with and within a package.

In saying that objects are 'packaged together', we mean that their classes are defined in the same package.

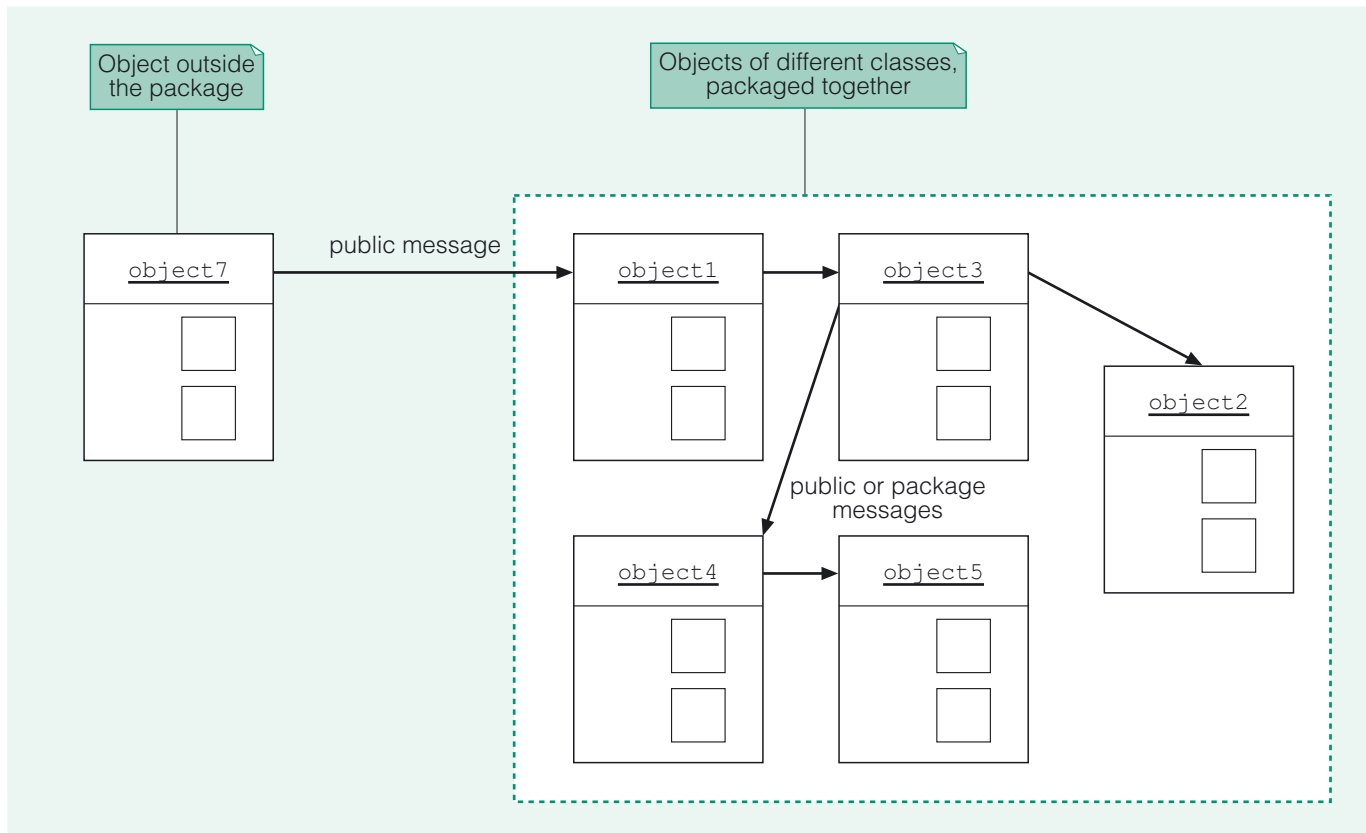


Figure 8 Interactions with and within a package

When developing a package, you should use access modifiers to enable appropriate interactions with and within the package, as well as providing appropriate data hiding. In other words:

- ▶ objects of classes within the package should be able to interact appropriately to provide the services required of the package;
- ▶ clients of the package should be able to interact with the package, obtaining its services;
- ▶ clients of the package should be protected from implementation changes within the package;
- ▶ the integrity of the package as a whole, and that of the individual objects within it, should be protected during its use by clients.

The **integrity** of a package refers to the consistency of the states of the objects in the package with any invariants on those objects.

To illustrate, recall the package `schoolcore`, which houses the core system classes within the School System. The package should respect the fact that each pupil is a member of a form. If a client were able to create a `Pupil` object without linking it to any `Form` object, then the integrity of the package would be violated: an invariant on the system would be broken

You looked at this package in Exercise 4 (Subsection 4.2).

SAQ 10

Suppose class `ClassA` is part of package `packagep`. How can the following be achieved?

- (a) `ClassA` has a method `methodOne`, which the developer of `packagep` wishes to use in implementing other classes within the package, without making the method available to client code outside the package.
- (b) `ClassA` has a method `methodTwo`, which the developer of the package wishes to make available to any client of `packagep`.

ANSWER.....

- (a) By giving `methodOne` package accessibility, that is, giving it no explicit access modifier (the default access modifier).
- (b) By giving `methodTwo` public accessibility.

To illustrate some of these ideas, we will now look at how access modifiers have been used within the School System, and at how its packages interact.

Exercise 6



In NetBeans, open the project `School`. Recall that it consists of two packages, `schoolcore` and `schoolgui`.

- (a) The core system package, `schoolcore`, is where the ‘underlying’ part of the system, representing forms and pupils and so on, is implemented.

Open the source code for the class `Pupil`, in `schoolcore`. This class has several members: instance variables, a constructor and methods. To what extent can each member be accessed outside the `Pupil` class?

- (b) Now open the source code for the class `SchoolGUI`, in the user interface package `schoolgui`. (There are two ‘views’ of this class: the Design view and the Source view. You should be looking at the Source view.) `SchoolGUI` is the sole class in this package. An object of the `SchoolGUI` class implements the user interface for the School System.

What statement imports the package `schoolcore` into the class definition of `SchoolGUI`?

Discussion.....

- (a) `Pupil` has two instance variables, `name` and `birthDate`, which both have private accessibility. They cannot be accessed by objects of any other class.

The constructor `Pupil(aName, aBirthDate)` has no access modifier, so it has package accessibility. This means that objects of any of the classes within the same package, `schoolcore`, can access this constructor. But classes in other packages, such as `schoolgui`, cannot.

The methods `getName()`, `getBirthDate()` and `toString()` have public accessibility. They can be accessed from any class, regardless of its location (so long as the class definition imports the `schoolcore` package, or at least the `Pupil` class from this package).

Note also that the `Pupil` class itself is declared as `public`. If a class is not declared as `public`, then access to its entire class definition is restricted to that class’s package: essentially the class is private to that package. This possibility will not need to be considered further.

- (b) The statement `import schoolcore.*;` at the beginning of the class definition.

We will now consider an example of the interaction of the School System packages, and the role of access modifiers. Because `SchoolGUI` imports the package `schoolcore`, a `SchoolGUI` object can access certain services provided by classes in the `schoolcore` package. It will need to access these services to handle user requests. Consider a situation where the user interface is requested by a user to enrol a new pupil into a form. What might the user interface do?

Do not be perturbed by the length of the code within `SchoolGUI`. The great majority of it has been automatically generated by NetBean’s GUI building facilities, which you will meet later in the course. You do not need to understand the implementation, except for small parts to which we will direct you.

We carried out a similar analysis of the handling of a user request, in Section 3 of *Unit 1*. The difference here is our consideration of access modifiers.

You saw in the last exercise that the class `Pupil` has a constructor, which initialises new `Pupil` objects. The class `Form` also has a useful-looking method, `addPupil(aPupil)`, which adds a pupil into a form.

The `Pupil` constructor and the `addPupil(aPupil)` method of `Form` would seem to have potential for use in enrolling a new pupil into a form. But the user interface, as an object of the class `SchoolGUI`, cannot access the `Pupil` constructor, because, as you saw in the last exercise, this constructor has package accessibility: it cannot be accessed by classes outside the `schoolcore` package. Code such as the following would not be valid inside `SchoolGUI`.

```
Pupil pupil = new Pupil(pupilName, pupilBirthDate);
```

Similarly, a `SchoolGUI` object cannot access the `Form` method `addPupil(aPupil)`, as this too has package accessibility. The following diagram illustrates the general situation.

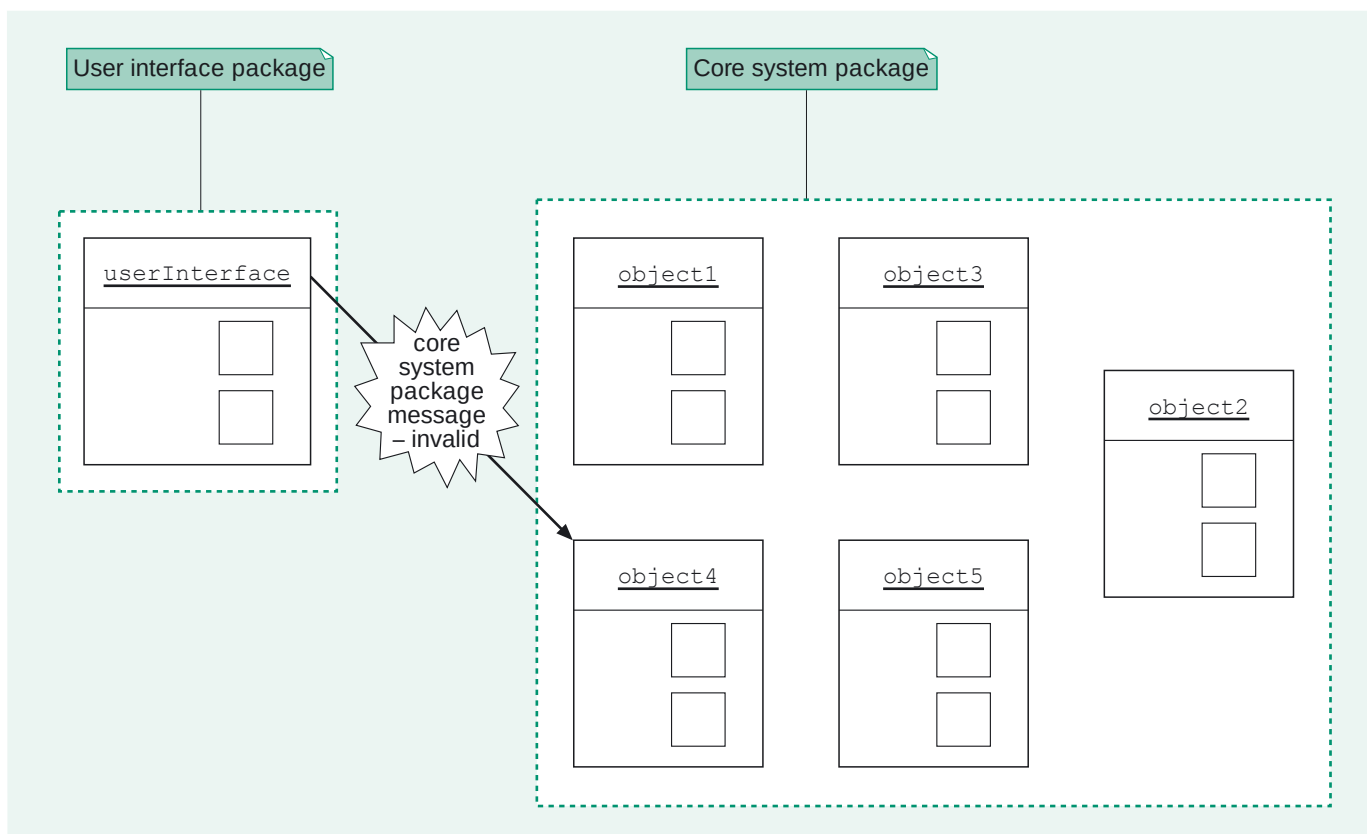


Figure 9 If a method of a class inside the package has package accessibility, then a client outside the package cannot access it

So how is the request to enrol a new pupil into a form handled? For this and, in fact, every task requested by a user, the user interface accesses the services of the core system through a `SchoolCoord` object, which is itself inside the core system. A user interface object, i.e. a `SchoolGUI` object, has an instance variable, `school`, which references, and enables it to send messages to, this `SchoolCoord` object. The `SchoolCoord` object is termed the **coordinating object**.

In the next exercise, you will investigate the role of the coordinating object in enrolling a new pupil into a form.

An alternative term for *coordinating object* is **orchestrating object**.

Exercise 7



Return to the `School` project in NetBeans.

- (a) Identify, at the beginning of the `SchoolGUI` code, the declaration of the instance variable `school`.
- (b) Go to the code for the method `doEnrolment()` of the class `SchoolGUI`. All you need to look at is the following code, which is positioned around the middle of the method code.

```
school.enrolPupil(nameField.getText(), date, theForm);
```

The expressions `nameField.getText()` and `date` respectively reference the pupil name and date of birth entered by the user, and `theForm` references the `Form` object corresponding to the form selected by the user.

Describe what service is being requested in this code, and of which object.

- (c) Explain, in terms of access modifiers, why it is valid for a `SchoolGUI` object to send the `SchoolCoord` object referenced by `school` the message `enrolPupil(nameField.getText(), date, theForm)`.
- (d) See that the code for the `SchoolCoord` method `enrolPupil(aName, aBirthDate, aForm)` is the following.

```
Pupil thePupil = new Pupil(aName, aBirthDate);
aForm.addPupil(thePupil);
```

Explain in terms of access modifiers:

- (i) why it is valid for a `SchoolCoord` object to invoke the `Pupil` constructor;
- (ii) why it is valid for a `SchoolCoord` object to send a `Form` object a message `addPupil(thePupil)`.

Discussion.....

- (a) The instance variable `school` is declared in the following line.

```
private SchoolCoord school;    /**the coordinating object*/
```

- (b) The coordinating object, referenced by `school`, is being requested to enrol a new pupil into a form.
- (c) The object referenced by `school` is of class `SchoolCoord`, whose method `enrolPupil(aName, aBirthDate, aForm)` has public accessibility, enabling it to be accessed by objects of any class, including `SchoolGUI`.
- (d) The `Pupil` constructor and the `Form` method `addPupil(thePupil)` are both accessible by a `SchoolCoord` object:
 - (i) the `Pupil` constructor has package accessibility, and `SchoolCoord` is in the same package as `Pupil`;
 - (ii) similarly, the `Form` method `addPupil(aPupil)` has package accessibility, and `SchoolCoord` is in the same package as `Form`.

In summary, the above exercise shows that the interactions between the packages `schoolgui` and `schoolcore` in response to a user request to enrol a new pupil into a form are as follows.

- The user interface sends a message requesting the coordinating object to enrol the new pupil into the form.
- In turn, the coordinating object invokes the `Pupil` constructor, initiating the creation of a new `Pupil` object, and then sends a message to a `Form` object to add the new pupil.

You will learn much more about this approach in Unit 6.

These interactions in fact exemplify the general form of interaction between the user interface and the core system in the School System. The approach can be outlined as follows.

- 1 The user interface sends a message to the coordinating object within the core system, requesting the core system's service in dealing with the user's task.
- 2 The coordinating object then coordinates the services provided by the objects in the core system to provide the service requested by the user interface.

Figure 10 illustrates this.

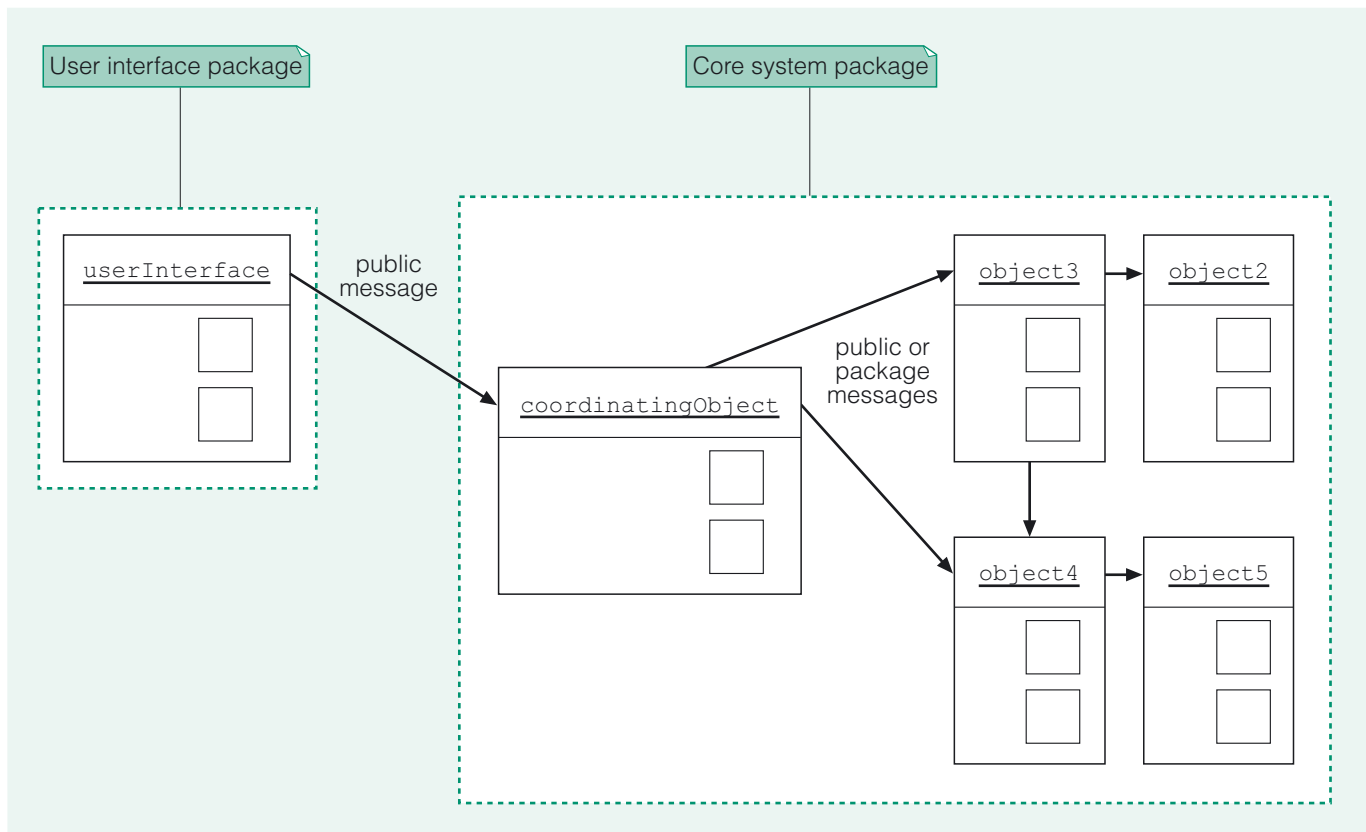


Figure 10 Interaction between the user interface and the core system

What are the benefits of this arrangement? In the particular case of enrolling a new pupil into a form, why not let the user interface have access to the `Pupil` constructor and the `Form` method `addPupil(aPupil)`? There are two main reasons.

- 1 Protecting the user interface from changes to the core system. Because the user interface sends its requests to the coordinating object, it can be protected from any changes to the implementation of the core system. The precise details of the coordinating object's response to a message – the actions it initiates within the core system – may change, but the user interface still sends the same message.
- 2 Preserving the integrity of the core system package. For example, if the user interface could invoke the `Pupil` constructor directly, it might create a `Pupil` object without linking it to a `Form` object, violating the integrity of the `schoolcore` package. The coordinating object's method `enrolPupil(aName, aBirthDate, aForm)`, to which the user interface does have access, both creates a `Pupil` object *and* ensures that it is linked to a `Form` object, so maintaining the integrity of the package. If a user interface has no choice but to call on the services of the coordinating object, then it is confined to interacting with the core system in controlled ways which do not damage the core system.

You have seen above that the integrity of the core system is better preserved if the user interface communicates with the core system via the coordinating object. However, the following exercise shows that there are situations in which it is appropriate (and safe) for the user interface to access other objects of the core system directly.

Exercise 8

The method `displayFormDetails()` of `SchoolGUI` includes the code `theForm.getName()`, where `theForm` references a `Form` object. (You need not look at the method.) The method `getName()` of `Form` has public accessibility, allowing its access in this way from outside the `schoolcore` package. But why should it be acceptable for this method to have public accessibility?

Discussion.....

No harm comes from allowing access to `getName()` from outside the `schoolcore` package. Revealing this simple attribute value, a `String`, poses no threat to the integrity of the core system.

In this section you have learnt how, in Java, access modifiers are used to achieve an appropriate level of data hiding for classes and packages. You also learnt about the advantages of using these facilities in designing the communication between a user interface package and a core system package. In the next section you will learn how, even using the techniques described above, one of these advantages, maintaining the integrity of the core system, can be compromised in certain situations.

6

Data hiding: privacy leaks

You have seen how access modifiers are used in protecting integrity by preventing inappropriate access to class members. In this section you will learn:

- ▶ that access modifiers alone are not always sufficient for complete protection;
- ▶ how careful coding can help prevent damage.

6.1 Compromising integrity

Having assigned private accessibility to the instance variables of a class, the developer has to decide whether and how clients outside that class can be allowed to manipulate the state of its objects. You have seen the example of the `Account` class with the public method `setBalance(anAmount)`, which enables clients to modify the balance whilst maintaining the invariant that the overdraft limit should not be exceeded. Such **setter methods** are common, and their usual purpose is exemplified by `setBalance(anAmount)`: to enable clients to change the value of an instance variable in a controlled way.

Getter methods, which allow clients to access the value of an instance variable, are also common. After all, simply requesting the value of an object's instance variable seems a harmless enough thing to do. In fact, as you will now see in the context of a variation on the School System, there is a hidden danger.

In the next exercise, you will look at a project called `CompromisedSchool`. It contains the same `schoolcore` package as the `School` project – except for one small change, which we will reveal shortly. However, there is no user interface package; instead, there is a package called `schoolclient` which contains a single class, `SchoolClient`. This class has a single static method, which is defined as follows.

Clients use the static method `getSchoolCoordObject()` to create a `SchoolCoord` object, because a `SchoolCoord` constructor is not publicly available. The reason for this is not relevant here.

```
public static void main(String[] args)
{
    //Set up a reference to a SchoolCoord object
    SchoolCoord school = SchoolCoord.getSchoolCoordObject();

    //print out the details of the SchoolCoord object
    System.out.println("School details are:");
    System.out.println(school);

    //get the forms
    Collection<Form> clientForms = school.getForms();

    //Now 'clear' the forms
    clientForms.clear();

    //again, print out the details of the SchoolCoord object
    System.out.println("School details are now:");
    System.out.println(school);
}
```

From the code and its comments, you can see that a `SchoolClient` object acts as a client, requiring the services of a `SchoolCoord` object. It creates a `SchoolCoord` object, prints out its details, sends it the message `getForms()`, and so on.

Exercise 9



In NetBeans, open the project `CompromisedSchool` which is in the folder `M256\M256Code\Exercises\Unit5`.

- (a) Open the source code for the class `SchoolClient` in the package `schoolclient`. What packages does `SchoolClient` import?
- (b) Now look at the source code for the class `SchoolCoord` in the package `schoolcore`.
 - (i) What does the instance variable `forms` reference? What is its accessibility?
 - (ii) Read the code for the method `getForms()`, and explain what this method does. What is its accessibility?

Discussion.....

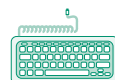
- (a) It imports the `schoolcore` package, via the `import schoolcore.*;` statement, and the `java.util` package, via the `import java.util.*;` statement.
- (b) In the class `SchoolCoord`:
 - (i) The instance variable `forms` references a collection of `Form` objects, representing all the forms in the school. It has private accessibility. This can be seen from the following declaration.

```
/** A collection of all the forms in the school.*/
private Collection<Form> forms;
```

- (ii) The method `getForms()` simply returns the value of the `forms` instance variable. It has public accessibility.

There is, on the face of it, nothing unusual about the situation described in the last exercise. The `SchoolCoord` object has a private instance variable, with a public method enabling a client – here a `SchoolClient` object – to get the value of this instance variable. The next exercise illustrates that in fact the integrity of a `SchoolCoord` object is compromised by this arrangement.

Exercise 10



Return to NetBeans and the project `CompromisedSchool`.

Consider the following `SchoolClient` code, which you saw above.

```
Collection<Form> clientForms = school.getForms();
clientForms.clear();
```

In the first line of code, a variable, `clientForms`, is set to reference the collection of `Form` objects that results from sending the message `getForms()` to the object referenced by `school`.

In the second line of code, a message `clear()` is sent to this collection of `Form` objects.

- (a) Use the specification of the `Collection` method `clear()` shown in Figure 11 to describe what you think the second line of code will achieve.

```
void clear()
    Removes all of the elements from this collection.
```

Figure 11 Specification of `clear()`

- (b) Run the project and consider the output. Describe the effect on the `SchoolCoord` object referenced by `school` of the lines of code in part (a) above.

Discussion.....

- (a) It looks as if the code should remove all the elements of the collection referenced by `clientForms`.
- (b) You should have found that the code causes the state of the `SchoolCoord` object referenced by `school` to change, as reported in the output description. In fact, it removes all of the forms from the school; that is, it empties the collection referenced by the `SchoolCoord` object's instance variable `forms`.

The above exercise illustrates a way for a client to access and modify an instance variable of a `SchoolCoord` object. The method `getForms()` returns the collection of `Form` objects to which the instance variable `forms` refers. The effect of the client code

```
Collection<Form> clientForms = school.getForms();
```

is to establish a new variable, `clientForms`, referencing this collection object. This external 'handle' on the collection object then enables the client to manipulate the collection without sending messages to the `SchoolCoord` object. Because the collection is part of the state of the `SchoolCoord` object, this means that the client can potentially invalidate the state of the `SchoolCoord` object: in our example, the client goes on to empty its collection of `Form` objects. The referencing involved is illustrated in the diagram below.

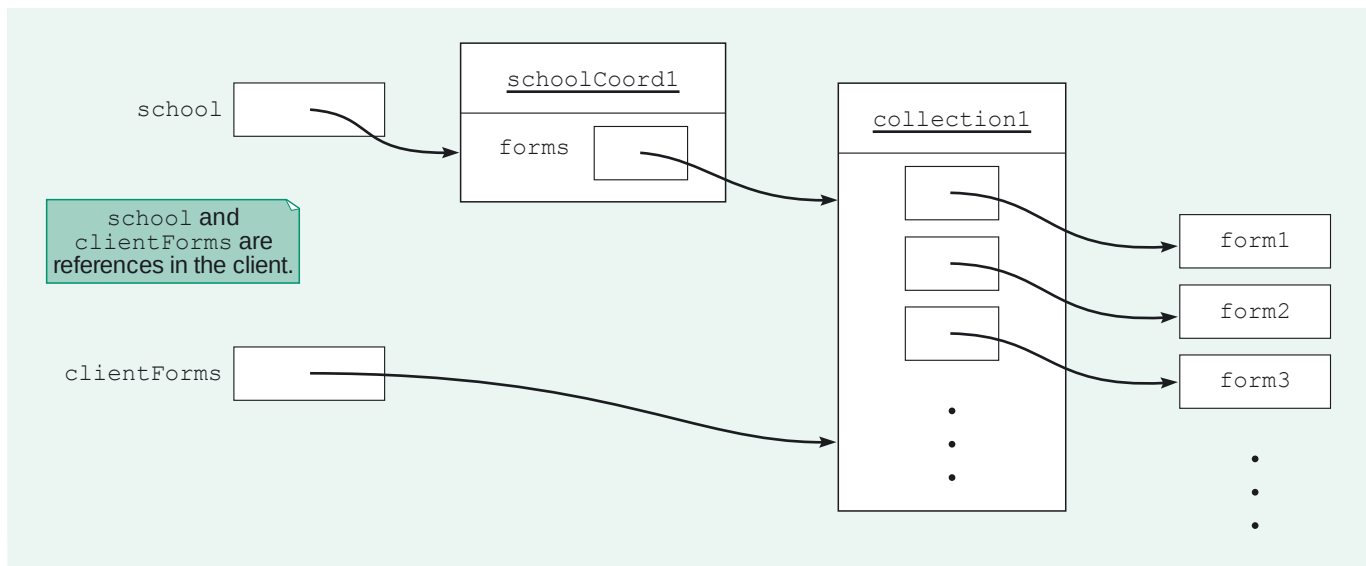


Figure 12 Two references to the same collection object

To summarise the situation: even when an instance variable is private, it may be possible for a client to access and improperly manipulate the state of an object referenced by the instance variable – a **privacy leak**. In general, there is potential danger whenever an instance variable references a **mutable object**. A mutable object is one whose state can be changed (an **immutable object** is one whose state cannot be changed). If a client can create a reference to a mutable object, then it can change its state. The mutable object in the above example is the collection of `Form` objects; the client exploited its mutability by removing all of its elements.

SAQ 11

Which of the following are mutable and which are immutable? Justify your answer.

- (a) The `String` object `"terry"`.
- (b) The `HashSet` object `{'t', 'e', 'r', 'y'}`.

ANSWER.....

- (a) Immutable. The state of a `String` object cannot be changed – no methods are provided to modify a `String` object's state.
- (b) Mutable. The state of a `HashSet` object can be changed in various ways. For example, if the variable `theSet` references the `HashSet` object `{'t', 'e', 'r', 'y'}`, then the following code adds an element to the `HashSet` object:

```
theSet.add('s');
```

6.2 Protecting mutable objects

The dangers of instance variables referencing mutable objects can be addressed by careful coding. If the purpose of a method is simply to provide information about the value of such an instance variable, the method may be coded to provide equivalent information in another way.

One way is for the method to create and return a **defensive copy** of a mutable object, that is, a distinct object with the same state. To understand the principle involved, consider the example of a web page. If a user accesses the Google home page, say, they can easily download and edit it. But of course they have just been supplied with a copy: Google would not want to let them have access to the original object.

Many predefined Java API classes have a particular kind of constructor called a **copy constructor**, which takes as argument an object of that class, and creates and returns a defensive copy of that object.

Exercise 11



Return to NetBeans and the `CompromisedSchool` project. Replace the code for the method `getForms()` in `SchoolCoord` with the following code.

```
return new HashSet<Form>(forms);
```

This constructor, `HashSet(aCollection)`, takes a collection as argument, and returns a new `HashSet` object containing the same elements as the collection.

Recompile and run the project.

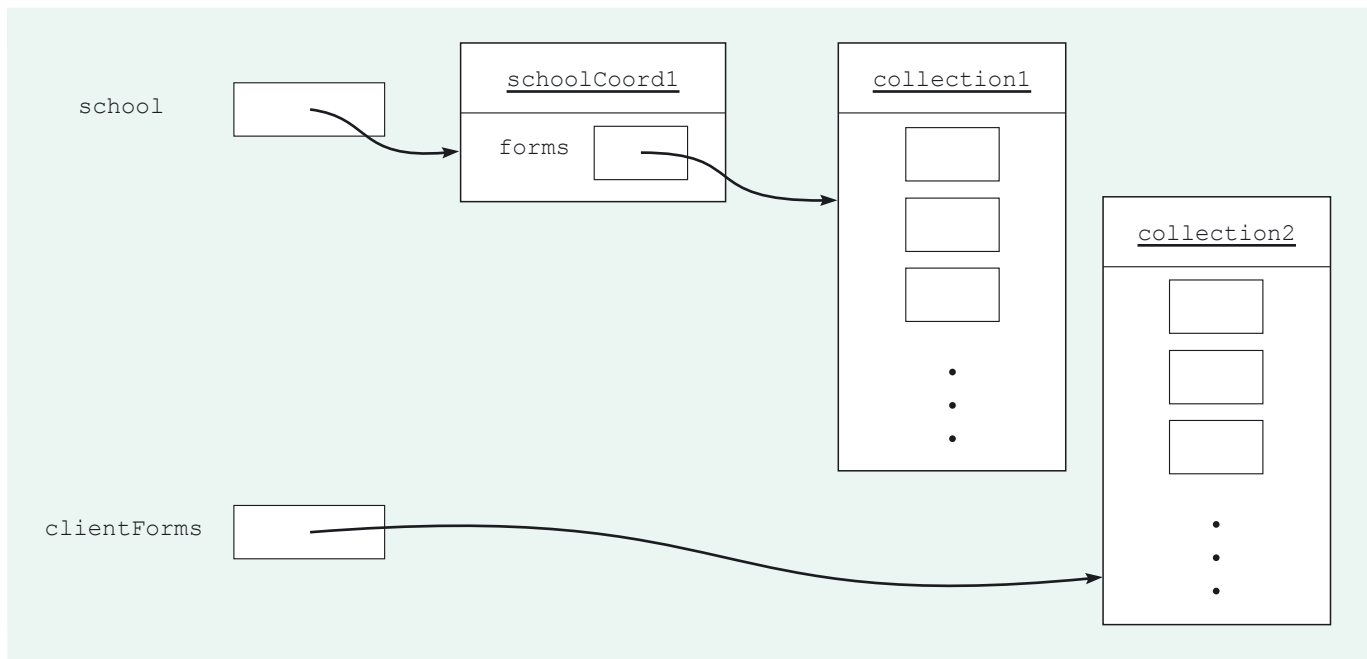
- (a) What now is the effect on the `SchoolCoord` object referenced by `school` of the following lines of code from `SchoolClient`?

```
Collection<Form> clientForms = school.getForms();
clientForms.clear();
```

- (b) Sketch a diagram like that of Figure 12 to illustrate the variable referencing involved.

Discussion.....

- (a) The code now has no effect on the `SchoolCoord` object referenced by `school`. The instance variable `clientForms` is not now set to reference the same collection object referenced by the instance variable `forms`. Instead it references a copy of this object, as shown in Figure 13. It is this copy that is emptied, not the original.
- (b) Figure 13 shows the variable referencing involved.



In fact, the situation is slightly more complex than illustrated in Figure 13 because the two distinct collection objects actually reference the same `Form` objects. To avoid this would require the more advanced concept of *deep copying*, which involves copying all objects. We will not be concerned with this issue, because it does not affect the integrity of our simple systems.

Figure 13 A reference to a defensive copy

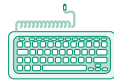
The concept of a privacy leak can be difficult to grasp. The scenario given below may help to clarify how it can arise and then cause a problem.

- ▶ Assume that `ClassA` has a public method, `method1`.
- ▶ Assume also that the invocation of `method1` on the `ClassA` object `objectA` returns a mutable object, `objectB`, which forms part of `objectA`'s state. (See Figure 12 for an example of this.)

The situation described here constitutes a privacy leak. It is a privacy leak because a client of `objectA` could use `method1` to create its own reference to `objectB` and change its (mutable) state. Because `objectB` forms part of the state of `objectA`, this may invalidate the state of `objectA`.

You have seen the use of a copy constructor to protect an object's state. The following exercise introduces you to another way to do this, which is the technique used in the original School System.

Exercise 12



Return to NetBeans and the `CompromisedSchool` project. Replace the code for the method `getForms()` in class `SchoolCoord` by the following code.

```
return Collections.unmodifiableCollection(forms);
```

Recompile and run the project.

(a) What now is the effect of the following lines of code from `SchoolClient`?

```
Collection<Form> clientForms = school.getForms();  
clientForms.clear();
```

(b) The static method `unmodifiableCollection(aCollection)` of the class `Collections` creates an ‘unmodifiable view’ of a collection `aCollection`. Essentially, it provides read-only access to the collection.

Use this fact to explain the effect you observed in part (a) above.

Discussion.....

- (a) An exception is thrown (an `UnsupportedOperationException`).
- (b) The exception is thrown in response to the attempt to remove all elements of the collection referenced by `clientForms`, since that collection cannot be modified.

This completes our discussion of data hiding in Java. In Section 5, you learnt that access modifiers could be used to control access to class members. However, in this section you saw that, in the presence of mutable objects, data hiding using access modifiers is not a foolproof way of maintaining the integrity of a system. In this case, other techniques of data hiding (defensive copies, read-only access) are required.

In the next section we conclude the unit by considering the use and production of components, and how this can fit into a software development process.

7

Using and producing components

Components embody desirable software qualities such as reusability and maintainability, but there are many issues to be addressed before the potential benefits of component-based software can actually be realised. There are two aspects of the component-based approach that need to be distinguished:

- 1 using existing components in creating a new piece of software (a process sometimes referred to as **design with reuse**);
- 2 producing components as part of developing new software, or specifically as marketable commodities (**design for reuse**).

In this section we discuss:

- ▶ design with and for reuse, and identify some of their advantages and challenges;
- ▶ how a component-based approach can be incorporated into a software development project;
- ▶ how M256 will use component-based development.

7.1 Design with reuse

Because we have not discussed any alternative, you may imagine that reusing a component means simply matching the services offered by the component with the needs of the project at hand, and employing the component as is. We need now to be explicit about two extensions to this wholesale reuse, each of which plays an essential part in using components. Our examples are of Java components, but the ideas are widely applicable throughout object-oriented programming.

The first alternative is reusing a component by creating a subclass of one of its classes – reuse by inheritance, that is, building on the behaviour provided by a class by adapting it or adding to it according to the needs of the current project. This technique should be very familiar to you, though you may not previously have thought of it as reuse.

The second is referred to as reuse by **composition**. Composition is based on including one object within the state of another. This apparently simple design technique, which you have seen many times in the systems you have explored so far on this course, is very useful in reusing components in certain situations. We will consider how it can be employed when the existing protocol of a component does not fit the requirements of the current project.

Composition

Software developers frequently find that an existing component does most of what they want, but simply cannot be fitted in with the other components as it stands, because its protocol is not quite right. The messages it recognises may all have different names from what is expected, for example.

How can the component still be reused in spite of this mismatch? Tinkering with its internals to change the message names is clearly a bad idea, because it may get 'broken', and this may not be an option anyway, since the component's user may not have access to its implementation. On the other hand, the other components cannot be expected to change what messages they send, because that would mean they would all have to be modified.

Consider this very simple example. A small drawing application called *Sketcher* is implemented as a single class which is designed to be incorporated into a larger system. So we can think of it as a component, albeit with quite a limited protocol, consisting of methods such as `clear()`, `drawLine()` and `drawCircle()`.

Suppose you wish to use *Sketcher* to develop a larger system, within which you are already committed to using a particular text-editor component, *TextEditor*.

TextEditor and *Sketcher* have been developed independently, by different developers. *TextEditor* has been constructed on the assumption that a drawing component will also be available, and that *TextEditor* will then be able to underline text by calling on the drawing component using a message named `underline()`. If you wish to 'plug together' *TextEditor* and *Sketcher*, you will have to reconcile the facts that *Sketcher* offers a line-drawing service via the message `drawLine()` whereas *TextEditor* expects to access such a facility via a message `underline()`.

A solution is to adapt the *Sketcher* component, using composition. This can be achieved by creating a new component – let's call it *AdaptedSketcher* – which has the protocol you do want. *AdaptedSketcher* has an instance variable, `theSketcher` say, referencing a *Sketcher* object, as illustrated in Figure 14.

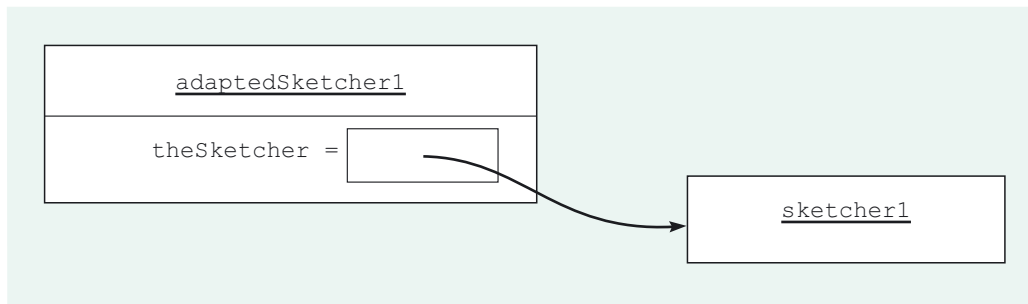


Figure 14 An Adapted Sketcher object references a Sketcher object

When an *AdaptedSketcher* object receives a message, it passes it on – *delegates* it – to its *Sketcher* object, as illustrated in the sequence diagram in Figure 15.

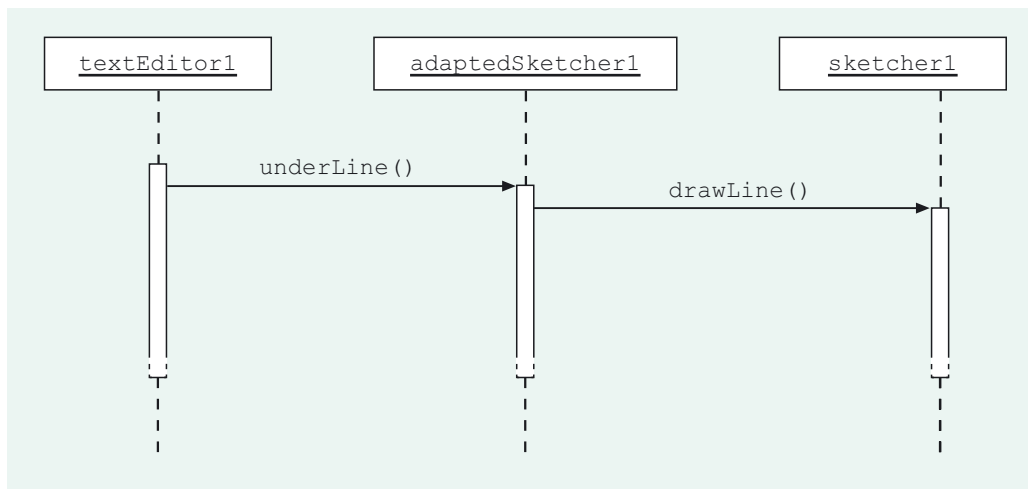


Figure 15 An AdaptedSketcher object delegates a message to its Sketcher object

The rest of the system sees only *AdaptedSketcher*, which has the expected protocol and fits in perfectly. But behind the scenes, *AdaptedSketcher* forwards any messages it is sent to its internal *Sketcher* object, which actually carries out the necessary actions in response. *Sketcher* is in fact being reused unchanged, but by wrapping it inside *AdaptedSketcher* it is made to fit with the rest of the system.

Exercise 13

Suppose the `Sketcher` message `clear()` *does* have a name appropriate for the rest of the system. Draw a sequence diagram illustrating communication between the `TextEditor`, `AdaptedSketcher` and `Sketcher` objects for this message.

Discussion.....

Figure 16 shows that the object, `adaptedSketcher1`, has a method, `clear()`, which simply calls the `clear()` method of the object `sketcher1`.

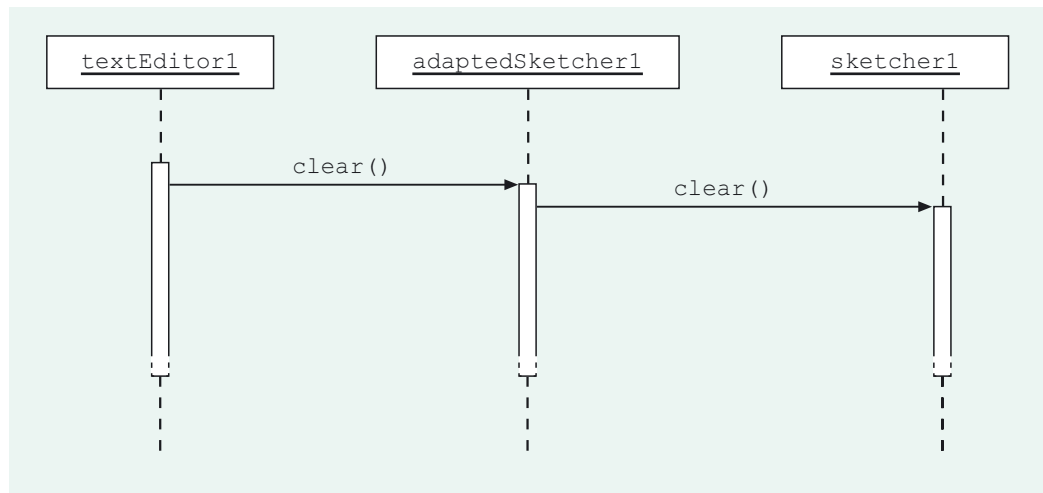


Figure 16 An `AdaptedSketcher` object delegates, unchanged, a message to its `Sketcher` object

In *Unit 9* you will consider some of the relative merits of composition and inheritance.

Composition is a mainstay of design. Along with inheritance, composition allows developers to reuse components to build powerful systems. Such opportunities for reuse have been found to occur time and time again in object-oriented programming.

Having reviewed a practical way in which components can be reused, it is time to reflect on the benefits and challenges of using components in creating new software.

Design with reuse: benefits

Our interest in components was motivated by the desire to create software with the qualities of maintainability and reusability. By definition, if software has been created using existing components, or if the components developed for this software have potential for reuse, then the software has good reusability. And component-based software is likely to have good maintainability, because an internal change to, or even complete replacement of, one component should not affect the rest of the system.

But there are other potential benefits in using components.

- ▶ Using components allows the expert knowledge of those who created them to be exploited by other developers.
- ▶ Using existing components in constructing new software may mean that the software can be produced more quickly and cheaply.
- ▶ Component-based software can be easier to test. If an existing component has already been tested, then all that may be required is to test its interaction with the rest of the system.
- ▶ Using components can increase software reliability. If a component has not only been formally tested as part of its original development, but has already been put to use in other software, problems with it are more likely to have been discovered and addressed.

Design with reuse: challenges

Components seem to hold such promise for software and its development that it may be hard to imagine any problems. But consider the fact that users of engineering components are able to choose from numerous off-the-shelf alternatives when selecting components to build a product. Software developers cannot yet select software components in the same fashion. There are companies specialising in producing components as marketable commodities, but except in very specialised areas, such companies are still in their infancy.

SAQ 12

What facilitates the design and construction of engineering products from components that may be manufactured by different companies?

ANSWER.....

Conformance of specifications to standards facilitates the design and construction of engineering products from components that may be manufactured by different companies.

A requirement for off-the-shelf software components to be a practical proposition is the adoption of industry standards for software components. Currently there are few common standards, and little adherence to those that exist. Different manufacturers are unlikely to produce components with compatible specifications.

OMG component standards

In *Unit 1*, we described the Object Management Group (OMG), the organisation which sets the UML standards. The OMG has made progress on specifying some kinds of software component. For example, there is an OMG specification of a surveillance component for use in air traffic control systems, which can interface with other surveillance processing systems.

SAQ 13

A component is to be replaced in a system by another component, which has similar functionality, but differences in its protocol. What technique might a developer employ to make this replacement viable?

ANSWER.....

Composition.

Aside from the lack of off-the-shelf components, there are other difficulties associated with using components.

- ▶ Finding a suitable component often relies on being able to find the products of previous projects. The search for a suitable component may be a lengthy process, especially if documentation of available components is poor.
- ▶ Something that may surprise you is that software developers can fight shy of using components. The challenge of creating something from scratch can be very attractive, and developers may resist using 'other people's' products.
- ▶ We previously suggested that using components decreases the testing load because a component will already have been tested. This is largely true, but

sometimes the original testing of the component may not have predicted and taken into account the kinds of future context in which it would be reused, and it may be necessary to test the component in the new context.

Ariane 5 – A failure to test reused software in a new context

On June 4, 1996, the maiden flight of the European Ariane 5 rocket launcher crashed approximately 40 seconds after take-off.

The CNES (French National Centre for Space Studies) and the European Space Agency investigated the crash, and concluded that it was the result of a software error. A process which should only have been applied to a number less than 2^{15} was instead erroneously applied to a larger number, resulting in the entire software system failing. The component performing the process was reused from the Ariane 4 software: the specification of this component was incomplete, and this led to the error.

7.2 Design for reuse

Components suitable for reuse do not simply emerge as a by-product of software development. Their creation requires anticipation of future needs, and is a job for skilled developers.

Design for reuse: benefits

Whether a particular component is actually reused depends on the services it offers. A component offering services required in a very specialised business area may not have services relevant to another area. If it is anticipated that a component may be reused in future projects, then it has the potential to contribute to those projects the benefits of design with reuse that have already been discussed.

But even if it is not envisaged that components produced during the development of a system will be used again in the future, taking a component-based approach and structuring software in terms of interacting components has significant advantages for software quality and for the software development process. It encourages an appropriate level of abstraction – a focus on specifying what services a component provides before considering how it might carry out these services. This has several desirable consequences.

- ▶ Once a component has been specified, it may be implemented largely independently from the rest of the software, and perhaps by different people. Further, different implementations can be investigated without affecting the rest of the development.
- ▶ A problem with how a component operates internally can be treated separately from a problem with what the component offers to the outside world. In other words, errors of implementation are separated from errors of specification. This makes errors easier to identify and fix and, because components should be independent from each others' implementations, the knock-on effects of internal errors are minimised.
- ▶ Once in operation, a component can be re-implemented or replaced, provided its specification is maintained.

Design for reuse: challenges

To have genuine potential for reuse, a component should have responsibility for providing a well-defined set of services. A component offering unrelated services may be too large, complex or inflexible to be easily incorporated into a system. At the same time, dependency between components must be carefully managed, because having to understand and incorporate relationships between components complicates the developer's job.

In understanding the choices to be made, the concepts of coupling and cohesion are relevant.

Cohesion is a measure of how strongly related and focused the **responsibilities** of a component are. High cohesion is considered desirable.

A component with low cohesion defines unrelated responsibilities, or defines too many responsibilities. The component may then be harder to reuse, because it is less likely to be useful in other projects, and it may be harder to understand, test and maintain.

Coupling is a measure of the extent to which a component is dependent on (coupled to) other components. Low coupling is generally considered desirable. A highly coupled component may be dependent on many other components, making it harder to reuse, because the components it relies on will also need to be present within any new system. And a highly coupled component may be hard to understand, test and maintain in isolation.

SAQ 14

Suppose that `ClassA` and `ClassB` are Java classes. Describe two ways in which `ClassA` may depend on `ClassB` in order to function.

ANSWER.....

Here are four ways we listed in Subsection 3.4; any two form a suitable answer.

- 1 `ClassA` may be a subclass of `ClassB`.
 - 2 `ClassA` objects may require a service provided by `ClassB` objects in carrying out a task. That is, inside one of `ClassA`'s methods, a message is sent to a `ClassB` object.
 - 3 `ClassA` may have a method that receives or returns a `ClassB` object.
 - 4 `ClassA` may define an instance variable referencing a `ClassB` object.
-

Coupling and cohesion are often considered in the context of large components, but they are also relevant to the design of single classes. Considering a class as a component, high cohesion means it should define responsibilities that you would expect to see carried out by a single object, and it should not have too many of them. Low coupling means that it should have low dependence on other classes.

SAQ 15

Why is coupling between classes inside a component of less concern than coupling between a class in the component and a class outside the component?

ANSWER.....

As the classes are part of the same component, they should be reused together anyway. The component's reuse potential is not diminished. Coupling between a class in the component and a class outside would mean that the class outside would have to be provided whenever the component was reused.

Object-oriented software is based on objects collaborating to achieve tasks – requesting services of each other. So some coupling is inevitable, and desirable. The developer's challenge is to balance issues of coupling and cohesion with other design concerns in creating useful and usable components. Whilst it is important to be aware of the principles of high cohesion and low coupling, the attainment of these ideals is not usually straightforward in practice. To illustrate the kinds of deliberation involved, the following exercise introduces a small example.

Exercise 14

Suppose you have developed an `Account` class having an instance variable `address`, designed to reference an instance of a class offering certain address-management services such as 'change house number'. Consider the following two possibilities for the component, `AccountManager`, within which the `Account` class is offered for reuse.

- 1 `Account` could be combined with a suitable class, `Address` say, inside `AccountManager`. Then whenever `Account` was required for reuse, the entire `AccountManager` component would be employed, so that `Account` could always make use of `Address`.
 - 2 `Account` could be packaged within `AccountManager`, but without the presence in `AccountManager` of a suitable `Address` class, thus requiring a suitable `Address` class to be made available in any system in which the `AccountManager` component is employed.
- (a) Which of 1 and 2 above is likely to result in `AccountManager` having lower coupling in a system in which it is reused?
 - (b) Which of 1 and 2 is likely to result in `AccountManager` having higher cohesion?

Imagine that a developer decides to use your `AccountManager` component in creating a banking system.

- (c) Suggest a possible advantage to the developer of the component being designed according to 1 above.
- (d) Suggest a possible advantage to the developer of the component being designed according to 2 above.

Discussion.....

- (a) 1 may result in lower coupling. `AccountManager` would not require the services of a separate address-related component.
 - (b) 2 may result in higher cohesion. `AccountManager` would have fewer responsibilities.
 - (c) The developer need not be concerned with locating and including a suitable `Address` facility.
 - (d) The developer might control the management of addresses, by being able to choose which `Address` class is used. For example, suppose that the `AccountManager` class requires addresses to be ordered in some way. The developer might have the flexibility to choose an `Address` class which orders addresses in alphabetical postcode order, or one that orders addresses in alphabetical street-name order.
-

7.3 A component-based approach

You learnt earlier that object-oriented software components have similarities to tangible components such as those used by engineers to build a vehicle. In fact, the similarities in the disciplines go further. In *Unit 1*, we introduced the term *software engineering* to refer to a systematic process of developing software. The term emphasises the similarities between systematic software development and the development of tangible products by engineers.

SAQ 16

What are the similarities between systematic software development and the development of tangible engineering products?

ANSWER.....

Successive activities are involved, moving from a general description of the software (product) through increasingly detailed designs (engineering blueprints) to the implementation (construction).

A common factor in many software development processes is that the focus of their early activities is on getting an overview of the system, concentrating on sketching its overall structure and not on small details. As the project progresses, so the details are filled in. This kind of approach is well suited to the use and production of components. The structure of the system is mapped out by specifying components and describing the services that each will provide so that they interact to fulfil the requirements of the system. Each component is then considered separately, determining its structure and behaviour in terms of smaller components, and so on, until the specified components can be implemented. At any stage, it may be discovered, by investigation or accident, that a suitable component already exists.

Incremental development

In *Unit 1*, we said that iterative development methods are particularly well suited to creating object-oriented software. An iterative method involves repetition of development phases, each iteration building on the outcomes of the previous one.

Iterative development can most easily facilitate a component-based approach when it is also *incremental*. **Incremental software development** means that the requirements of the software are partitioned from the start, with the partitions being developed separately. Partitions may be developed in parallel by different groups of people, or one partition could be developed at each iteration.

SAQ 17

How might incremental development contribute towards the development of components?

ANSWER.....

A partition comprising related requirements may be developed into a distinct 'piece' of the system with a well-defined purpose, that is, into a component.

With judicious partitioning of requirements, a significant advantage of incremental development is that the initial set of requirements can lead to an early prototype, often corresponding to key components of the system. Such a prototype may not be

discarded, but instead go through successive modification and augmentation until it meets all the requirements. This is **incremental prototyping**. The prototypes developed along the way to the final system consist of more and more components, developing increasingly sophisticated functionality. Prototypes may in fact be handed over to the client at various stages in **multiple releases** of the software.

Use cases and components

The development concepts and techniques that we introduce in this course, in common with many object-oriented development processes, are based on the concept of a use case. A use case, as you learnt in *Unit 2*, expresses a requirement of the system. For example, one of the use cases of the Hospital System is summarised as *Admit Patient*, and involves admitting a new patient to a ward. In use case-based development, the requirements can be partitioned to give sensible groupings of use cases, so that appropriate components can be specified.

To illustrate, imagine that the requirements of our Hospital System are more complex, and in particular that, in addition to *Admit Patient* (to a ward), there are several other use cases concerned with admitting a patient, such as:

- ▶ *Admit Day Patient*
- ▶ *Admit Accident and Emergency Patient.*

Grouping all such requirements together, the developers might specify (and then develop or, if they were fortunate, purchase) a component – let's call it `AdmissionsManager` – which would provide services to help fulfil these requirements.

SAQ 18

Suppose another component specified within the hospital system is `BedManager`, which deals with occupying and vacating beds in wards. When might `AdmissionsManager` require the services of `BedManager`?

ANSWER.....

In admitting a patient to a ward, the `AdmissionsManager` component may need to call on the `BedManager` component to identify a suitable vacant bed, and then occupy it.

7.4 Component-based development in M256

In *Unit 6*, we will resume consideration of our Hospital System example, through which you are learning about principles and practices of software development. Remember that rather than advocating a particular development method, we are introducing concepts and techniques common to many methods, and showing how they could be applied in developing example systems, such as the Hospital System. How will the principles of component-based development influence our application of these ideas?

The necessary simplicity of our systems means that the existing components we use are mainly the predefined Java API packages. And we will not have cause to develop components of the complexity of those mentioned above (`AdmissionsManager` and so on). That would be beyond the scope of a course at this level.

Our approach nevertheless includes many elements of a component-based approach. You have seen that, regardless of whether a desired outcome is to produce components useful to future developments, there can be great benefits in taking on board the principles of the approach.

You may want to refer to the *Handbook* for a reminder of the Hospital System. However, you do not need to recall its details here.

SAQ 19

What is meant by a component-based approach to software development?

ANSWER.....

A component-based approach to software development is an approach based on the philosophy of reusing existing components, and of structuring the software into interacting components which may be reused.

A primary feature of the approach in M256 is that we develop the user interface and the core system as separate components. In so doing, we enable our core system component to interact with different user interfaces. But we must also protect its integrity during these interactions. We will also want the implementation of our components to be such that future implementation changes are straightforward. In achieving these goals, we will make careful use of the object-oriented and Java-specific features discussed in this unit (see the following SAQ).

SAQ 20

Which features introduced in this unit, and falling under the following headings, contribute to component-based software?

- (a) Object-oriented features
- (b) Java-specific features

ANSWER.....

(a) Object-oriented features include:

- ▶ encapsulation;
- ▶ data hiding;
- ▶ protocols;
- ▶ polymorphism.

(b) Java-specific features include:

- ▶ packages;
 - ▶ access modifiers;
 - ▶ techniques for countering privacy leaks (defensive copies, read-only access).
-

Of immediate concern in *Units 6* and *7* will be making 'good' design decisions. How should we choose between different ways of achieving a task within our system? You will see that the concepts of coupling and cohesion can be useful in deciding which of two designs is preferable.

In this section, you have learnt of the benefits and challenges of reuse in relation to both design *with* reuse and design *for* reuse. You have also seen how an iterative and incremental development method facilitates a component-based approach, and how use cases can evolve into components. Finally you were presented with an outline of how the principles of component-based development will be applied in the software development you will meet in subsequent units.

8

Summary

In this unit you met two aspects of software quality. One is a successful software development process (exhibiting qualities such as timeliness and affordability); the other is a successful 'finished' product, that is, the software itself (exhibiting qualities such as reusability and maintainability). You discovered that the idea of developing software using components holds considerable, but as yet unfulfilled, promise for improving both aspects.

You found that certain object-oriented concepts, notably encapsulation and data hiding, are integral to producing and using object-oriented components, and you revised and extended your knowledge of how Java deals with these concepts.

A component-based approach, involving design with and for reuse, is considered to be one of the main ways forward in increasing the quality both of software development and of software itself, and you were introduced to some of the challenges, but also the benefits, that adopting such an approach brings.

LEARNING OUTCOMES

After studying this unit you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ give examples of aspects of software quality;
- ▶ analyse and discuss the key ideas involved in software maintenance;
- ▶ analyse and discuss the key ideas involved in software reuse;
- ▶ reason about particular examples involving software components;
- ▶ describe the characteristics that make classes/objects suitable software components;
- ▶ explain the role played by specifications in the construction and use of components, and make appropriate use of component specifications;
- ▶ identify dependencies between components, and explain how these dependencies affect a component's construction and use;
- ▶ use and document Java packages;
- ▶ use Java access modifiers to define classes whose objects exhibit appropriate data hiding;
- ▶ use Java access modifiers to enable appropriate interaction between packages;
- ▶ identify potential privacy leaks, and use different coding techniques to prevent them;
- ▶ reuse a simple class using composition, and illustrate this reuse using sequence diagrams;
- ▶ identify the key issues in following a component-based approach to software development;
- ▶ analyse the coupling and cohesion of simple components;
- ▶ outline how a component-based approach can fit into a software development process.

Glossary

access modifier One of the keywords `public`, `protected` and `private`, specifying the extent to which class members are accessible from outside the class. In the absence of one of these keywords the default package access applies.

application programming interface (API) In M256 this term refers to the predefined Java packages. More generally, the services offered by a component may be referred to as the component's API.

behaviour (of an object) An object's set of responsibilities.

class member An element of a class definition, such as an instance variable, method or constructor.

cohesion A measure of how strongly related and focused the responsibilities of a component are. High cohesion is considered desirable.

component On this course, unless indicated to the contrary, taken to mean an object-oriented software component.

component-based approach An approach to software development based on the philosophy of reusing existing components, and of structuring the software into interacting components which may be reused.

component-based software Software created using a component-based approach.

composition Reusing a class through an instance of the class being part of the state of an object of a new class.

coordinating object The core system object through which the user interface accesses the services of the core system.

copy constructor A constructor that returns a copy of its argument, i.e. a different object of the same class with the same state.

coupling A measure of the extent to which a component is dependent on other components. Low coupling is considered desirable.

data hiding Protecting an object's implementation details by restricting access to them.

decompiler A tool that takes compiled Java files and derives the source code.

defensive copy (of an object) A distinct object with the same state as the original.

design for reuse Producing components as part of developing new software, or specifically as marketable commodities.

design with reuse Using existing components in creating a new piece of software.

efficiency A measure of how well a system carries out its tasks.

encapsulation The packaging together in an object of data and the operations that can be applied to that data.

engineering components The tangible artefacts designed and constructed by engineers.

getter method A method enabling clients to discover the value of an instance variable.

immutable object An object whose state cannot be changed.

implementation details (of a class) How the class is coded, including the code for its methods and constructors, and the declaration of its instance variables.

incremental prototyping Successively modifying and augmenting a prototype until it meets all the requirements.

incremental software development An approach to software development where the requirements of the software are partitioned and partitions developed separately.

integrity (of a package) The consistency of the states of the objects in the package with any invariants on the objects.

integrity (of an object) The validity of an object's state, that is, the consistency of the values of its instance variables with any invariants on the object.

JAR file A form of zip file used to distribute compiled Java code, documentation and other information required for software to be used.

maintainability The extent to which software is easily maintained.

multiple releases (of software) Prototypes handed over to the client at various stages of the software's development.

mutable object An object whose state can be changed.

object-oriented software component One or more classes grouped together in some way to form a piece of software that has a well-defined purpose and that can be combined with other pieces of software to construct a larger system.

orchestrating object See coordinating object.

package A Java mechanism for grouping classes together to form an object-oriented software component.

polymorphism The capability for objects of different classes to respond to the same message in a manner appropriate for each object.

portability A measure of how easily a system can run on different platforms.

post-condition What is achieved by invoking a method, if its pre-conditions are satisfied.

pre-condition A restriction on the circumstances under which a method may be invoked.

privacy leak A situation where it may be possible for a client to access and improperly manipulate the state of an object.

protocol (of a class) The specifications of the methods defined in a class, together with the specifications of inherited methods, which define the corresponding messages that its objects will respond to.

protocol (of a component) The combined protocols of the component's classes, defining how clients can interact with the component.

responsibility (of a component) The services that the component offers.

responsibility (of an object) The tasks, or services, carried out by the object on receipt of appropriate messages.

reusability A measure of the extent to which existing artefacts have been used during the development of the software in question, and the extent to which the software itself offers artefacts for reuse.

setter method A method enabling clients to change the value of an instance variable in a controlled way.

software component A piece of software with a well-defined purpose that can be combined with other pieces of software to construct a larger system. On this course, unless the context indicates otherwise, the term is taken to mean object-oriented software component.

specification (of a method) A description of the purpose and usage of a method, including its pre-conditions and post-conditions.

specification (of an engineering component) A description of the engineering component's interface, requirements and services.

specification (of an object-oriented software component) A description of how to interact with the component, and of its requirements and services.

testability A measure of how easily a system can be tested.

unnamed package The default Java package for a user-defined class.

References

Sauer, C. and Cuthbertson, C. (2004) *The State of IT Project Management in the UK 2002–2003* [online], ComputerWeekly.com, <http://www.computerweeklyms.com/pmsurveyresults> (Accessed 21 December 2005).

Index

- A
 - access modifier 26
- accessibility 26
 - package 27
 - private 27
 - protected 27
 - public 27
- affordability 5
- application programming interface (API) 21
- B
 - behaviour (of an object) 16
- C
 - class member 26
- COBOL 9
- cohesion 47
- component 15
- component-based approach 12
- component-based software 12
- composition 42
- coordinating object 32
- copy constructor 39
- core system 24
- coupling 47
- D
 - data hiding 16
- decompiler 25
- default access modifier 27
- defensive copy 39
- delegate 43
- design
 - for reuse 42
 - with reuse 42
- E
 - efficiency 7
- encapsulation 15
- engineering component 12
- G
 - getter method 36
- I
 - immutable object 38
- implementation 19
- implementation details 15
- incremental
 - prototyping 50
 - software development 49
- independence (of classes) 16
- inheritance 42
- integrity
 - of a package 30
 - of an object 16
- interface 13
- J
 - JAR file 25
- L
 - legacy system 9
- M
 - maintainability 5, 7
- maintenance 7
- method specification 22–23
- multiple releases 50
- mutable object 38
- O
 - Object Management Group (OMG) 45
- orchestrating object 32
- P
 - package 21
- polymorphism 19
- portability 7
- post-condition 23
- pre-condition 22
- privacy leak 38
- protocol
 - of a class 15
 - of a component 20
- R
 - reliability 5
- responsibility
 - of a component 47
 - of an object 16
- reusability 6, 10
- S
 - setter method 36
- software component
 - general 6, 12
 - object-oriented 20
- software engineering 49
- specification
 - of a class's protocol 19
 - of a method 15, 22–23
 - of an engineering component 13
 - of an object-oriented software component 20
- T
 - testability 7
- timeliness 5
- U
 - unmodifiable 41
- unnamed package 21
- use case 50
- usefulness 5